

Study Guide: Artificial Intelligence — A Modern Approach (Russell & Norvig)

Generated 2026-06-03. Subject: Computer Science / AI.
Exam format: Mixed (MCQ, short answer, essay, problem-solving). Coverage: comprehensive (no length limit).

Chapter-by-Chapter Breakdown

Ch. 1 — Introduction

Named Entities (Terms & Definitions)

Term	Definition
AI	Study of agents that receive percepts and perform actions
Rational agent	Acts to achieve best expected outcome
Standard model	Agents that do the right thing based on fixed objective
Value alignment problem	Human objectives must align with machine objectives
Provably beneficial AI	Machines uncertain about human objectives
Turing test	Requires NLP, KR, automated reasoning, ML
Total Turing test	Adds vision and robotics

Approaches to AI (Four-way classification)

Approach	Goal	Method
Acting humanly	Turing test	NLP, KR, reasoning, ML
Thinking humanly	Cognitive modeling	Introspection, psychology experiments, brain imaging
Thinking rationally	Laws of thought	Logic, syllogisms
Acting rationally	Rational agent approach	Prevailing standard

Foundations Disciplines

Discipline	Key Names / Ideas
Philosophy	Aristotle (syllogisms), dualism vs materialism, empiricism, logical positivism
Mathematics	Boole (logic), Frege (quantifiers), Gödel (incompleteness), Turing (computability), Church-Turing thesis, NP-completeness, tractability
Economics	Decision theory, game theory, operations research, satisficing (Simon)
Neuroscience	Neurons, Broca's area, EEG, fMRI, optogenetics, brain-machine interfaces
Psychology	Behaviorism, cognitive psychology, cognitive science
Computer engineering	Moore's law, GPU, TPU, quantum computing
Control theory	Cybernetics, Wiener, feedback, cost function
Linguistics	Chomsky (transformational grammar), computational linguistics

History Milestones

Year	Event
1943	McCulloch & Pitts — artificial neurons, Hebbian learning, SNARC
1950	Turing test proposed
1956	Dartmouth Conference, Logic Theorist (Newell & Simon), GPS, physical symbol system hypothesis
1958	McCarthy — Lisp, Advice Taker, resolution method
1960s	Microworlds (blocks world, SAINT, STUDENT, ANALOGY)
1969	Perceptrons (Minsky & Papert) — limitations led to AI winter
1970s	Expert systems (DENDRAL, MYCIN, R1), certainty factors, frames
1980s	AI winter, back-propagation resurgence, connectionist models

Year	Event
1988	Bayesian networks (Pearl), reinforcement learning + MDPs
2011+	Big data, deep learning, ImageNet, AlphaGo, AlphaZero

State of the Art Applications Robotic vehicles, legged locomotion, planning/scheduling, machine translation, speech recognition, recommendations, game playing, image understanding, medicine, climate science.

Risks & Benefits

- Lethal autonomous weapons
- Surveillance
- Biased decisions
- Employment impact
- Safety-critical system failures
- Cybersecurity vulnerabilities
- King Midas problem, gorilla problem, superintelligence
- Assistance games, inverse reinforcement learning

Ch. 2 — Intelligent Agents

Named Entities (Terms & Definitions)

Term	Definition
Agent	Anything that perceives its environment through sensors and acts upon it through actuators
Percept	The content an agent's sensors perceive at a given moment
Percept sequence	Complete history of everything the agent has ever perceived
Agent function	Mathematical mapping from percept sequence to action
Agent program	Concrete implementation of the agent function

Term	Definition
Rational agent	For each percept sequence, selects action expected to maximize performance measure, given evidence from percepts and built-in knowledge
Consequentialism	Evaluating behavior by its consequences
Performance measure	Evaluates any sequence of environment states
Omniscience	Knowing actual outcome of actions (impossible)
Information gathering	Actions to modify future percepts
Autonomy	Learning to compensate for partial/incorrect prior knowledge
Task environment	The “problem” to which rational agents are the “solutions”
PEAS	Performance, Environment, Actuators, Sensors
Softbot	Software robot in virtual environments
Agent architecture	Computing device with sensors/actuators; agent = architecture + program
Simple reflex agent	Selects actions based only on current percept
Condition-action rule	if condition then action
Model-based agent	Maintains internal state using transition + sensor models
Transition model	Knowledge about how the world works
Sensor model	Knowledge about how state is reflected in percepts
Goal-based agent	Has goal information describing desirable situations
Utility function	Maps states to utility values
Expected utility	Utility expected on average given probabilities and utilities
Learning element	Component for making improvements
Performance element	Component for selecting external actions
Critic	Provides feedback relative to fixed performance standard
Problem generator	Suggests exploratory actions
Model-free agent	Learns best action without learning how action changes environment
Atomic representation	Each state indivisible, black box

Term	Definition
Factored representation	State splits into fixed set of variables/attributes
Structured representation	State includes objects with attributes and relationships
Localist representation	One-to-one mapping between concepts and memory locations
Distributed representation	Concept spread over many memory locations
Environment class	Set of environments from a distribution
Open-loop system	Ignores percepts during execution
Closed-loop system	Monitors percepts during execution

Processes / Algorithms Rationality depends on four things:

1. Performance measure 2. Agent's prior knowledge 3. Actions the agent can perform 4. Percept sequence to date

Definition of rational agent: For each percept sequence, select action expected to maximize performance measure, given evidence from percept sequence and built-in knowledge.

TABLE-DRIVEN-AGENT: Persistent percepts sequence + table; doomed — taxi has $> 10^{600,000,000,000}$ entries for 1 hour.

SIMPLE-REFLEX-AGENT: $state \leftarrow INTERPRET-INPUT(percept)$,
 $rule \leftarrow RULE-MATCH(state, rules)$, $action \leftarrow rule.ACTION$

MODEL-BASED-REFLEX-AGENT: $state \leftarrow UPDATE-STATE(state, action, percept, transition\ model, sensor\ model)$

General Learning Agent: Four components: learning element, performance element, critic, problem generator

Hierarchies / Classifications Four basic agent kinds (increasing complexity): 1. Simple reflex agents — respond directly to percepts 2. Model-based reflex agents — maintain internal state 3. Goal-based agents — consider future consequences 4. Utility-based agents — maximize expected utility

Representation axis: Atomic $\rightarrow$ Factored $\rightarrow$ Structured

PEAS examples: Taxi driver, Medical diagnosis, Satellite image analysis, Part-picking robot, Refinery controller, Interactive English tutor

Comparisons / Trade-offs

Property	Easier/Harder
Fully observable vs. partially observable	Fully obs. is easier
Single-agent vs. multiagent	Single-agent is simpler
Deterministic vs. nondeterministic	Deterministic is easier
Episodic vs. sequential	Episodic is much simpler
Static vs. dynamic	Static is easier
Discrete vs. continuous	Discrete is easier
Known vs. unknown	Known is easier

Hardest case: Partially observable, multiagent, nondeterministic, sequential, dynamic, continuous, unknown.

Rationality vs. omniscience: Rationality = maximize expected performance; omniscience = maximize actual performance (impossible).

Formulas & Equations **Lookup table size:** $\sum_{t=1}^T |P|^t$ entries, P = set of possible percepts, T = lifetime

Rules, Laws & Theorems **Wiener's warning:** "The purpose put into the machine is the purpose which we really desire" — King Midas problem.

Performance measure design rule: Design according to what one actually wants achieved, not according to how one thinks the agent should behave.

Edge Cases / Traps

- **King Midas problem:** Getting exactly what you ask for, not what you want
- **Dung beetle:** Assumes ball of dung exists; fails silently when removed
- **Sphex wasp:** Innate plan fails; cannot learn to adapt
- **Table-driven approach** is doomed: impossible table size

Empirical Evidence

- Chess lookup table: $\geq 10^{150}$ entries (vs observable universe atoms $< 10^{80}$)
- Taxi camera: ~ 70 MB/sec $\rightarrow > 10^{600,000,000,000}$ entries for 1 hour

People

Person	Contribution
Norbert Wiener	Warning about purpose/machine alignment
Aristotle	Nicomachean Ethics — practical reasoning
McCarthy (1958)	“Programs with Common Sense”
Turing (1950)	Learning machines
Samuel (1959, 1967)	Checkers learning program
Newell & Simon (1972)	Human Problem Solving
Pearl (1988)	First AI text covering probability and utility

Design Paradigms

- Agent = architecture + program
- First step: specify task environment using PEAS
- Exploration vs. exploitation: problem generator suggests suboptimal actions for learning

Ch. 3 — Solving Problems by Searching

Named Entities (Terms & Definitions)

Term	Definition
Problem-solving agent	Plans ahead; considers action sequences forming a path to goal
Search	Computational process of finding a path to a goal
Goal formulation	Adopting the goal; limits objectives and actions
Problem formulation	Devising description of states and actions
Solution	Sequence of actions reaching the goal
State space	Set of possible environment states
Initial state	Starting state
Goal states	States the agent aims to reach
Action	ACTIONS(s) returns applicable actions in s
Transition model	RESULT(s,a) returns state from doing a in s
Action cost function	$c(s,a,s')$ — numeric cost
Path	Sequence of actions
Optimal solution	Solution with lowest path cost
Abstraction	Removing detail; key to tractability

Term	Definition
Node	Corresponds to a state in search tree; includes parent, action, path-cost
Expand	Applying ACTIONS to a state, generating children
Frontier	Set of nodes generated but not yet expanded
Reached	States that have had a node generated
Best-first search	Chooses node minimizing evaluation function $f(n)$
Evaluation function	$f(n)$ — determines node selection order
Completeness	Guaranteed to find solution if one exists
Cost optimality	Finds solution with lowest path cost
Depth d	Number of actions in optimal solution
Branching factor b	Number of successors of a node
Breadth-first search	Expands root, then all successors, level by level
Uniform-cost search	Best-first search with $f(n) = \text{path-cost}$; Dijkstra's algorithm
Depth-first search	Always expands deepest node first
Backtracking search	Generates one successor at a time; $O(m)$ memory
Depth-limited search	DFS with depth limit ℓ
Iterative deepening search	Tries all depth limits $0, 1, 2, \dots$
Bidirectional search	Forward from initial, backward from goal
Informed search	Uses heuristic function $h(n)$
Heuristic function $h(n)$	Estimated cost from n to goal
Greedy best-first search	$f(n) = h(n)$
A* search	$f(n) = g(n) + h(n)$; optimal with admissible heuristic
Admissible heuristic	Never overestimates cost to goal
Consistency / Monotonicity	$h(n) \leq c(n, a, n') + h(n')$
Optimally efficient	A* expands minimal nodes for given heuristic
Pruning	Eliminating possibilities without examining them
Satisficing	Accepting "good enough" suboptimal solutions
Weighted A* search	$f(n) = g(n) + W \times h(n)$; $W > 1$

Term	Definition
Beam search	Limits frontier size to k best nodes
IDA*	Iterative deepening with f-cost cutoff
RBFS	Recursive best-first search; linear-space A*
SMA*	Simplified memory-bounded A*
Effective branching factor b^*	b^* such that uniform depth-d tree has $N+1$ nodes
Domination	h_2 dominates h_1 if $h_2(n) \geq h_1(n)$ for all n
Relaxed problem	Problem with fewer restrictions; cost of optimal solution is admissible heuristic
Pattern database	Stores exact solution costs for every subproblem instance
Disjoint pattern databases	Non-overlapping subproblems whose costs can be summed
Landmark point	Precomputed optimal path costs to chosen vertices
Differential heuristic	$h_{DH}(n) = \max_L$

Processes / Algorithms Four-phase problem-solving: Goal formulation → Problem formulation → Search → Execution

Search problem (5 components): States, Initial state, Goal states (via IS-GOAL), Actions (ACTIONS(s)), Transition model (RESULT(s,a)), Action cost ($c(s,a,s')$)

BEST-FIRST-SEARCH:

```

node ← NODE(STATE = problem.INITIAL)
frontier ← priority queue ordered by f, with node
reached ← lookup table with INITIAL → node
while not IS-EMPTY(frontier):
    node ← POP(frontier)
    if IS-GOAL(node.STATE): return node
    for each child in EXPAND(problem, node):
        s ← child.STATE
        if s not in reached or child.PATH-COST < reached[s].PATH-
COST:
            reached[s] ← child
            add child to frontier
return failure

```

BREADTH-FIRST-SEARCH: FIFO queue; test goal on generation

UNIFORM-COST-SEARCH: BEST-FIRST-SEARCH with PATH-COST as f

ITERATIVE-DEEPENING-SEARCH:

```
for depth = 0 to  $\infty$ :  
    result  $\leftarrow$  DEPTH-LIMITED-SEARCH(problem, depth)  
    if result  $\neq$  cutoff: return result
```

A* search: $f(n) = g(n) + h(n)$; expands nodes with smallest f first

Weighted A*: $f(n) = g(n) + W \times h(n)$

RBFS: Recursive; tracks f -limit (best alternative path f -value); backs up best leaf f -values

Comparisons / Trade-offs

Criterion	BFS	UCS	DFS	DLS	IDS	Bidirectional
Complete?	Yes ¹	Yes ^{1,2}	No	No	Yes ¹	Yes ^{1,4}
Optimal cost?	Yes ³	Yes	No	No	Yes ³	Yes ^{3,4}
Time	$O(b^d)$	$O(b^{\lceil 1 + \lceil C^*/\epsilon \rceil \rceil})$	$O(b^m)$	$O(b^l)$	$O(b^d)$	$O(b^{\lceil d/2 \rceil})$
Space	$O(b^d)$	$O(b^{\lceil 1 + \lceil C^*/\epsilon \rceil \rceil})$	$O(bm)$	$O(b^l)$	$O(bd)$	$O(b^{\lceil d/2 \rceil})$

¹complete if b finite and solution exists; ²if costs $\geq \epsilon > 0$; ³if all action costs identical; ⁴if both directions BFS/UCS

Formulas & Equations

- **BFS total nodes:** $1 + b + b^2 + \dots + b^d = O(b^d)$
- **UCS complexity:** $O(b^{\lceil 1 + \lceil C^*/\epsilon \rceil \rceil})$
- **DFS memory:** $O(bm)$
- **IDS total nodes:** $N(\text{IDS}) = (d)b^1 + (d-1)b^2 + \dots + b^d = O(b^d)$
- **Bidirectional motivation:** $b^{\lceil d/2 \rceil} + b^{\lceil d/2 \rceil} \ll b^d$
- **Consistency:** $h(n) \leq c(n, a, n') + h(n')$
- **A* surely expanded:** $f(n) < C$; **A* never:** $f(n) > C$
- **Effective branching factor:** $N + 1 = 1 + b^* + (b^*)^2 + \dots + (b^*)^d$
- **Composite heuristic:** $h(n) = \max\{h_1(n), \dots, h_k(n)\}$
- **Differential heuristic:** $h_{DH}(n) = \max_L |C(n, L) - C(\text{goal}, L)|$
- **8-puzzle state space:** $9!/2 = 181,440$; **15-puzzle:** $16!/2 > 10$ trillion

Rules, Laws & Theorems

- **Admissibility theorem:** With admissible heuristic, A* is cost-optimal
- **Consistency theorem:** Every consistent heuristic is admissible; with consistent heuristic A* is cost-optimal AND first time reaching a state is on optimal path

- **Optimal efficiency theorem:** A* with consistent heuristic is optimally efficient
- **Relaxed problem theorem:** Cost of optimal solution to relaxed problem = admissible heuristic
- **Domination principle:** If $h_2 \geq h_1$, A* with h_2 never expands more nodes than A* with h_1
- **Negative-cost cycle warning:** If any cycle has negative cost, optimal solution is infinite

A* Admissibility Proof Assume A* returns $C > C^*$ (suboptimal). Then $\exists$ node n on optimal path, unexpanded: - $f(n) > C^*$ (otherwise n would be expanded) - $f(n) = g(n) + h(n) \leq g(n) + h(n) = C$ Contradiction: $f(n) > C^*$ and $f(n) \leq C$. $\therefore$ A returns only optimal paths.

People

Person	Contribution
Dijkstra (1959)	Shortest-path algorithm
Hart, Nilsson, Raphael (1968)	A* algorithm
Newell & Simon (1957, 1961)	Logic Theorist, GPS
Korf (1985b)	IDA*
Korf (1993)	RBFS
Pearl (1984)	Heuristics, consistency proof
Pohl (1970, 1971)	Weighted A*, bidirectional search
Culberson & Schaeffer (1996, 1998)	Pattern databases
Korf & Felner (2002)	Disjoint pattern databases
Goldberg et al. (2006)	Landmarks + bidirectional A* for Microsoft Maps

Design Paradigms

- Problem relaxation $\rightarrow$ admissible heuristic (supergraph property)
- Pattern databases: precompute subproblem costs via backward search + DP
- Landmarks: precompute $C^*(v, L)$; differential heuristic is admissible
- Metalevel state spaces: learn to search better
- Coarse-to-fine search: hierarchical abstraction

Ch. 4 — Search in Complex Environments

Named Entities (Terms & Definitions)

Term	Definition
Local search	Search from start state to neighbors without tracking paths or reached states
Optimization problem	Find best state according to objective function
State-space landscape	States with elevation = value of objective function
Hill climbing	Finding highest peak; gradient descent when minimizing cost
Steepest ascent	Move to neighbor with highest value
Local maximum	Peak higher than neighbors but lower than global max
Ridge	Sequence of local maxima difficult for greedy algorithms
Plateau	Flat area (flat local max or shoulder)
Shoulder	Plateau from which uphill progress possible
Sideways move	Move to equal-value neighbor
Stochastic hill climbing	Chooses randomly from uphill moves
First-choice hill climbing	Generates successors randomly until better found
Random-restart hill climbing	Series of hill climbs from random starts; complete with prob 1
Simulated annealing	Hill climbing + random walk; accepts bad moves with prob $e^{-\{\Delta E/T\}}$ decreasing with T
Local beam search	Keeps track of k states; generates all successors; selects k best
Stochastic beam search	Chooses successors with probability proportional to value
Genetic algorithm	Individuals as strings; selection, crossover, mutation
Evolution strategies	Individuals as real-number sequences
Genetic programming	Individuals as computer programs
Crossover point	Random point to split and recombine parent strings
Mutation rate	Probability each bit is flipped
Elitism	Keeping top-scoring parents in next generation
Schema	Substring with unspecified positions
Baldwin effect	Learning relaxes fitness landscape, accelerating evolution
Gradient ∇f	Vector of steepest slope direction

Term	Definition
Step size α	Small constant for gradient update
Line search	Doubling α until f starts to decrease
Newton-Raphson method	$x \leftarrow x - g(x)/g'(x)$
Hessian matrix $H_f(x)$	Matrix of second derivatives
Convex optimization	Convex constraint region + convex objective; polynomially solvable
Belief state	Set of physical states the agent believes possible
Conditional plan	Contingency plan specifying actions based on percepts
AND-OR search tree	OR nodes (agent choices) and AND nodes (environment outcomes)
Cyclic solution	Solution with loops
Sensorless / Conformant problem	Agent receives no percept information
Coercion	Agent forces world to goal through actions without sensing
Monitoring / Filtering	Maintaining belief state over time
Localization	Working out location given map, percepts, and actions
Online search	Interleave computation and action
Competitive ratio	Ratio of actual path cost to optimal if known
Dead end	State from which no goal is reachable
LRTA*	Learning Real-Time A*; updates $H(s)$ based on experience
Optimism under uncertainty	Assume untried actions lead to goal with minimal cost

Processes / Algorithms **HILL-CLIMBING:**

```

current ← problem.INITIAL
while true:
    neighbor ← highest-valued successor
    if VALUE(neighbor) ≤ VALUE(current): return current
    current ← neighbor

```

SIMULATED-ANNEALING:

```

for t = 1 to ∞:

```

```

T ← schedule(t)
if T = 0: return current
next ← random successor
ΔE ← VALUE(current) - VALUE(next)
if ΔE > 0: current ← next
else: current ← next with probability  $e^{\{-\Delta E/T\}}$ 

```

GENETIC-ALGORITHM:

```

repeat:
  weights ← WEIGHTED-BY(population, fitness)
  population2 ← empty
  for i = 1 to SIZE(population):
    p1, p2 ← WEIGHTED-RANDOM-CHOICES(population, weights, 2)
    child ← REPRODUCE(p1, p2)
    if small random prob: child ← MUTATE(child)
    add child to population2
  population ← population2
until fit enough or time elapsed

```

AND-OR-SEARCH: OR-SEARCH (for each action, call AND-SEARCH on RESULTS); AND-SEARCH (must succeed for all states)

Belief-state transition (sensorless): Deterministic: $b' = \{s' : s' = \text{RESULT_P}(s,a), s \in b\}$; Nondeterministic: $b' = \cup \text{RESULTS_P}(s,a)$

Prediction-Observation-Update cycle: 1. Predict: $\hat{b} = \text{RESULT}(b,a)$ 2. Possible percepts: $\text{POSSIBLE-PERCEPTS}(\hat{b}) = \{o : o = \text{PERCEPT}(s), s \in \hat{b}\}$ 3. Update: $b_o = \text{UPDATE}(\hat{b}, o) = \{s : o = \text{PERCEPT}(s), s \in \hat{b}\}$

LRTA*-AGENT: Updates $H[s]$ after leaving a state; selects action minimizing $c(s,a,s') + H[s']$; optimism under uncertainty

Comparisons / Trade-offs Local vs systematic search: Local: little memory, reasonable solutions in large spaces, not systematic. Systematic: complete, more memory.

Hill climbing on 8-queens: Without sideways: 14% solved, 86% stuck, avg 4 steps. With sideways (100 limit): 94% solved, avg 21 steps.

Simulated annealing vs hill climbing vs random walk: Hill climbing efficient but stuck; random walk complete but extremely inefficient; SA combines both.

Newton-Raphson vs gradient ascent: NR uses Hessian (second derivatives), can jump directly; gradient ascent first-order, slower but cheaper per step.

Sensorless vs conditional planning: Sensorless can coerce world; conditional branches on percepts.

Formulas & Equations

- **Gradient ascent:** $x \leftarrow x + \alpha \nabla f(x)$
- **Newton-Raphson:** $x \leftarrow x - H_f^{-1}(x) \nabla f(x)$
- **8-queens state space:** $8^8 \approx 17$ million
- **Airport siting:** $f(x) = \sum_i \sum_{c \in C_i} [(x_i - x_c)^2 + (y_i - y_c)^2]$

Rules, Laws & Theorems

- **Schema theorem:** If average fitness of schema instances is above mean, number of instances grows over time
- **Optimism under uncertainty:** Untried actions assumed to lead to goal with cost $h(s)$
- **Dead end theorem:** No algorithm can avoid dead ends in all state spaces
- **Random walk completeness:** Complete on finite safely explorable spaces

People

Person	Contribution
Newton (1671) / Raphson (1690)	Newton-Raphson method
Darwin (1859)	On the Origin of Species
Holland (1975)	Genetic algorithms
Kirkpatrick et al. (1983)	Simulated annealing
Metropolis et al. (1953)	Metropolis algorithm
Korf (1990)	LRTA*
Kantorovich (1939) / Dantzig (1949)	Linear programming

Ch. 5 — Adversarial Search and Games

Named Entities (Terms & Definitions)

Term	Definition
Adversarial search	Search in competitive environments with conflicting goals
Perfect information	Fully observable; players know entire state

Term	Definition
Zero-sum game	One player's gain equals another's loss
MAX / MIN	MAX maximizes utility, MIN minimizes
TO-MOVE(s)	Player whose turn it is
ACTIONS(s)	Set of legal moves
RESULT(s,a)	Transition model
IS-TERMINAL(s)	Terminal test
Utility function	Final numeric value to player at terminal state
Game tree	Complete search tree to terminal states
Ply	One move by one player
Minimax value	Utility for MAX assuming both play optimally
Alpha-beta pruning	Eliminates subtrees that don't affect outcome
α	Best (highest) choice for MAX along path
β	Best (lowest) choice for MIN along path
Transposition table	Cache of evaluated state values
Heuristic evaluation function (EVAL)	Estimates utility of a state
Cutoff test	Replaces terminal test at depth limit
Weighted linear function	$EVAL(s) = \sum w_i f_i(s)$
Quiescence search	Extra search on nonquiescent positions
Horizon effect	Damage pushed beyond search depth via delaying tactics
Monte Carlo tree search (MCTS)	Selection $\rightarrow$ Expansion $\rightarrow$ Simulation $\rightarrow$ Back-propagation
UCT (UCB1 applied to trees)	$UCB1(n) = U(n)/N(n) + C \times \sqrt{(\log N(PARENT(n)) / N(n))}$
Expectiminimax	Minimax generalized for games with chance nodes
Belief state	Set of all logically possible board states consistent with percept history

Processes / Algorithms **MINIMAX-SEARCH:**

```

MAX-VALUE returns (utility, move):
  if TERMINAL: return UTILITY(state, player), null
  v  $\leftarrow$  - $\infty$ 
  for each a in ACTIONS(state):
    v2, a2  $\leftarrow$  MIN-VALUE(RESULT(state, a))
    if v2 > v: v, move  $\leftarrow$  v2, a
  return v, move

```


- Time: $O(b^m)$, Space: $O(bm)$ or $O(m)$
- Chess: $b \approx 35$, $m \approx 80 \rightarrow 35^{80} \approx 10^{123}$ states

ALPHA-BETA-SEARCH: - Same as minimax but with α, β bounds; prune when $v \geq \beta$ (MAX) or $v \leq \alpha$ (MIN) - Perfect ordering: $O(b^{(m/2)})$; Random ordering: $O(b^{(3m/4)})$

Monte Carlo Tree Search: 1. **Selection:** Choose moves by selection policy to leaf 2. **Expansion:** Generate a new child 3. **Simulation:** Playout from child using playout policy 4. **Back-propagation:** Update all nodes on path to root

Expectiminimax:

```

if TERMINAL(s): return UTILITY(s, MAX)
if TO-MOVE(s) = MAX: max_a EXPECTIMINIMAX(RESULT(s,a))
if TO-MOVE(s) = MIN: min_a EXPECTIMINIMAX(RESULT(s,a))
if TO-MOVE(s) = CHANCE:  $\sum_r P(r)$  EXPECTIMINIMAX(RESULT(s,r))

```

- Complexity: $O(b^m n^m)$ where n = distinct chance outcomes

Formulas & Equations

- **Minimax value:** $\text{MAXIMAX}(s) = \text{UTILITY}(s, \text{MAX})$ if TERMINAL; $= \max_a \text{MINIMAX}(\text{RESULT}(s,a))$ if MAX; $= \min_a \text{MINIMAX}(\text{RESULT}(s,a))$ if MIN
- **Heuristic minimax:** $\text{H-MINIMAX}(s,d) = \text{EVAL}(s, \text{MAX})$ if IS-CUTOFF; else max/min over actions
- **Weighted linear:** $\text{EVAL}(s) = w_1 f_1(s) + \dots + w_n f_n(s)$
- **UCB1:** $\text{UCB1}(n) = U(n)/N(n) + C \times \sqrt{(\log N(\text{PARENT}(n)) / N(n))}$
- **Alpha-Beta complexity:** Best $O(b^{(m/2)})$; Random $O(b^{(3m/4)})$

Comparisons / Trade-offs

Criterion	Alpha-Beta	MCTS
Evaluation	Uses heuristic EVAL	Average of playout outcomes
Works well	Small b	High b
Error	Single miscalculation can mislead	Aggregate of many playouts
Game type	Chess (low b , good EVAL)	Go (high b , hard EVAL)

People

Person	Contribution
Zermelo (1912)	Minimax algorithm
Shannon (1950)	Type A/B strategies

Person	Contribution
McCarthy (1956)	Conceived alpha-beta
Knuth & Moore (1975)	Proved alpha-beta correctness
Pearl (1982b)	Asymptotic optimality of alpha-beta
Kocsis & Szepesvári (2006)	UCT selection mechanism
Silver et al. (2016, 2018)	AlphaGo, AlphaZero
Schaeffer (2007)	Solved checkers
Brown & Sandholm (2017, 2019)	Libratus, Pluribus

Case Studies **Tic-tac-toe:** $< 9! = 362,880$ terminal nodes; utilities: win +1, loss -1, draw 0 **Chess:** $b \approx 35$, $m \approx 80$; AlphaZero defeated Stockfish 155-6-839 **Go:** branching factor 361; AlphaGo defeated Lee Sedol 4-1, Ke Jie 3-0 **Backgammon:** $n = 21$ distinct rolls; TD-Gammon reached world champion level **Poker:** Libratus used abstraction + overnight hole-plugging; Pluribus defeated 6-player pros

Ch. 6 — Constraint Satisfaction Problems

Named Entities (Terms & Definitions)

Term	Definition
CSP	Problem with variables (X), domains (D), constraints (C)
Variables (X)	Set $\{X_1, \dots, X_n\}$
Domains (D)	Set of allowable values per variable
Constraints (C)	(scope, rel) specifying allowable combinations
Assignment	Set $\{X_i=v_i, X_j=v_j, \dots\}$
Consistent assignment	Does not violate any constraints
Complete assignment	Every variable assigned
Solution	Consistent, complete assignment
Constraint graph	Nodes = variables, edges = binary constraints
Binary CSP	Only unary and binary constraints
Alldiff constraint	All variables must have different values
Constraint propagation	Using constraints to reduce legal values
Node consistency	All domain values satisfy unary constraints

Term	Definition
Arc consistency	For every X , $\exists$ value in D_i satisfying binary constraint
Path consistency	Tightens binary constraints using triples
K-consistency	For any $k-1$ variables with consistent assignment, $\exists$ consistent value for k th
Backtracking search	DFS with partial assignments; exploits commutativity
MRV heuristic	Choose variable with fewest legal values
Degree heuristic	Choose variable with most constraints on unassigned variables
Least-constraining-value	Choose value that rules out fewest choices for neighbors
Forward checking	After assignment, establish arc consistency for assigned variable
MAC (Maintaining Arc Consistency)	After assignment, run AC-3 on neighboring arcs
Conflict-directed backjumping	Backtrack to most recent assignment in conflict set
Constraint learning	Record minimal conflict subset as no-good
Min-conflicts heuristic	Choose value minimizing conflicts
Tree-structured CSP	Any two variables connected by only one path
Cycle cutset	Subset whose removal makes graph a tree
Tree decomposition	Transform graph into tree where node = set of variables
Tree width	(Largest node size) - 1

Processes / Algorithms **AC-3 (Arc Consistency):**

```

queue ← all arcs
while queue not empty:
    ( $X_i, X_j$ ) ← POP(queue)
    if REVISE(csp,  $X_i, X_j$ ):
        if size  $D_i = 0$ : return false
        for each  $X_k$  in  $X_i$ .NEIGHBORS -  $\{X_j\}$ : add ( $X_k, X_i$ ) to queue
return true

```

Complexity: $O(c d^3)$ where c = binary constraints, d = max domain size

BACKTRACKING-SEARCH:

```

BACKTRACK(csp, assignment):
    if complete: return assignment
    var ← SELECT-UNASSIGNED-VARIABLE(csp, assignment)
    for each value in ORDER-DOMAIN-VALUES(csp, var, assignment):
        if consistent:
            add {var=value} to assignment
            inferences ← INFERENCE(csp, var, assignment)
            if inferences ≠ failure: add inferences, recurse
            remove
    return failure

```

MIN-CONFLICTS:

```

current ← initial complete assignment
for i = 1 to max_steps:
    if current is solution: return current
    var ← randomly chosen conflicted variable
    value ← value minimizing CONFLICTS(csp, var, v, current)
    set var = value in current
return failure

```

TREE-CSP-SOLVER: $O(n d^2)$ — linear in variables

Cutset conditioning: $O(d^c \cdot (n-c) d^2)$ where c = cutset size

Comparisons / Trade-offs

Technique	Benefit	Drawback
Node consistency	Eliminates unary violations	Doesn't propagate
AC-3	Propagates across binary constraints	Can't detect all inconsistencies
Forward checking	Fast incremental arc consistency	Doesn't look ahead far enough
MAC	Full propagation	More expensive per node
MRV	Fail-first; prunes early	Needs tie-breaker

Approach	Complexity	Memory
Cutset conditioning	$O(d^c \cdot (n-c)d^2)$	Linear
Tree decomposition	$O(n d^{(w+1)})$	Exponential in w
Plain backtracking	$O(d^n)$	$O(n)$
Tree-structured CSP	$O(n d^2)$	$O(n)$

Formulas & Equations

- **CSP:** $X = \{X_1, \dots, X_n\}$, $D = \{D_1, \dots, D_n\}$, $C = \{\text{scope, rel}\}$
- **AC-3:** $O(c \cdot d^3)$
- **Tree CSP:** $O(n \cdot d^2)$
- **Cutset:** $O(d^c \cdot (n-c) \cdot d^2)$
- **Tree decomposition:** $O(n \cdot d^{(w+1)})$, w = tree width
- **Cryptarithmic:** TWO + TWO = FOUR: column equations with carries C_1, C_2, C_3

Rules, Laws & Theorems

- CSP is NP-complete in general
- Strong n -consistency $\rightarrow O(n^2 \cdot d)$ solution
- Tree-structured CSPs solvable in linear time $O(n \cdot d^2)$
- Graphs with bounded tree width solvable in polynomial time
- Four-color theorem (Appel & Haken, 1977)
- Every finite-domain CSP can be reduced to binary CSPs

People

Person	Contribution
Montanari (1974)	CSP as general class
Mackworth (1977)	AC-3 algorithm
Waltz (1975)	Constraint propagation for vision
Freuder (1978-1985)	k -consistency, tree-structured CSPs
Dechter (1990)	Cycle-cutset, constraint learning
Gaschnig (1977, 1979)	Backjumping, backmarking
Prosser (1993)	Conflict-directed backjumping
Minton et al. (1992)	Min-conflicts heuristic

Design Paradigms

- Factored representation: state = variable assignments
 - Constraint propagation as inference
 - Backtracking + inference interleaving
 - Fail-first principle (MRV)
 - Fail-last principle (least-constraining-value)
 - Cutset conditioning: remove nodes to get tree
 - Tree decomposition: collapse variable sets
-

Ch. 7 — Logical Agents

Named Entities (Terms & Definitions)

Term	Definition
Knowledge-based agent	Uses reasoning over internal knowledge representation
Knowledge base (KB)	Set of sentences
Sentence	Expression in KR language
TELL / ASK	Operations to add/query KB
Inference	Deriving new sentences from old
Wumpus world	Cave + rooms + wumpus + pits + gold; agent testbed
Syntax	Structure of well-formed sentences
Semantics	Meaning/truth w.r.t. possible worlds
Model	Mathematical abstraction with truth values
Satisfaction	Model m satisfies sentence α : $m(\alpha) = \text{true}$
Entailment ($\models$)	$\alpha \models \beta$ iff $M(\alpha) \subseteq M(\beta)$
Model checking	Enumerating all models to check entailment
Soundness	Derives only entailed sentences
Completeness	Can derive any entailed sentence
Propositional logic	Logic with proposition symbols and connectives
Literal	Atomic sentence or negated atomic sentence
Conjunction ($\wedge$)	"And" — conjuncts
Disjunction ($\vee$)	"Or" — disjuncts
Implication ($\Rightarrow$)	Premise $\rightarrow$ conclusion
Biconditional ($\Leftrightarrow$)	"If and only if"
Validity	True in all models (tautology)
Satisfiability	True in some model
Resolution	Sound inference rule; complete algorithm
Clause	Disjunction of literals
CNF	Conjunction of clauses
Horn clause	At most one positive literal
Definite clause	Exactly one positive literal
Forward chaining	Data-driven reasoning from facts
Backward chaining	Goal-directed reasoning from query
DPLL	Complete backtracking SAT algorithm
Unit clause	Assign unit clause's literal before branching
heuristic	
Pure symbol	Symbol always same sign in all clauses

Term	Definition
WALKSAT	Local search SAT algorithm
Successor-state axiom	Defines F^{t+1} in terms of F^t and actions
Frame problem	Need to specify what stays unchanged after action
Qualification problem	Cannot specify all exceptional preconditions

Processes / Algorithms **KB-AGENT:** TELL(KB, percept) → ASK(KB, action-query) → TELL(KB, action) → return action

TT-ENTAILS? (Truth-table enumeration): Enumerate all models; check if α true in all KB models. Complexity $O(2^n)$.

PL-RESOLUTION:

```

clauses ← CNF of KB ∧ ¬α
while true:
    for each pair  $C_i, C_j$  in clauses:
        resolvents ← PL-RESOLVE( $C_i, C_j$ )
        if empty clause: return true
        new ← new ∪ resolvents
    if new ⊆ clauses: return false
    clauses ← clauses ∪ new

```

CNF Conversion: 1. Eliminate $\Leftrightarrow$, 2. Eliminate $\Rightarrow$, 3. Move $\neg$ inwards, 4. Distribute $\vee$ over $\wedge$

DPLL:

```

if every clause true: return true
if some clause false: return false
P,value ← FIND-PURE-SYMBOL(symbols, clauses, model)
if found: recurse with P=value
P,value ← FIND-UNIT-CLAUSE(clauses, model)
if found: recurse with P=value
P ← FIRST(symbols)
return DPLL(clauses, rest, model ∪ {P=true}) OR DPLL(clauses, rest, model ∪ {P=false})

```

Improvements: early termination, pure symbol, unit propagation.

WALKSAT:

```

model ← random assignment
for i = 1 to max_flips:
    if model satisfies clauses: return model
    clause ← randomly selected false clause

```

```

    if RANDOM(0,1) ≤ p: flip random symbol in clause
    else: flip symbol maximizing satisfied clauses
return failure

```

Sound but not complete (can't detect unsatisfiability).

SATPLAN: Translate to SAT → SAT solver → Extract plan Requirements: Init₀, transition axioms, goal, precondition axioms, action exclusion axioms

Hybrid Wumpus Agent: TELL(KB, percept) → ASK(KB, OK) → plan route → find gold/shoot wumpus/climb out

Formulas & Equations

- **Entailment:** $\alpha \models \beta$ iff $M(\alpha) \subseteq M(\beta)$ iff $(\alpha \Rightarrow \beta)$ valid iff $(\alpha \wedge \neg \beta)$ unsatisfiable
- **Successor-state axiom:** $F^{t+1} \Leftrightarrow \text{ActionCauses}F^t \vee (F^t \wedge \neg \text{ActionCausesNot}F^t)$
- **Resolution:** $\ell_1 \vee \dots \vee \ell_k, m_1 \vee \dots \vee m_n$ with ℓ_i and m_j complementary → resolvent

Truth Tables

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
F	F	T	F	F	T	T
F	T	T	F	T	T	F
T	F	F	F	T	F	F
T	T	F	T	T	T	T

Clause Types

Type	Positive literals	Example
Definite clause	Exactly one	$\neg A \vee \neg B \vee C$
Horn clause	At most one	$\neg A \vee \neg B \vee C$ or $\neg A \vee \neg B$
Goal clause	None	$\neg A \vee \neg B$
Unit clause	One literal	P

Rules, Laws & Theorems

- **Modus Ponens:** $\alpha \Rightarrow \beta, \alpha \vdash \beta$
- **Monotonicity:** If $KB \models \alpha$ then $KB \wedge \beta \models \alpha$
- **Deduction theorem:** $\alpha \models \beta$ iff $(\alpha \Rightarrow \beta)$ is valid
- **Refutation:** $\alpha \models \beta$ iff $(\alpha \wedge \neg \beta)$ unsatisfiable

- **Ground resolution theorem:** If S unsatisfiable, $RC(S)$ contains empty clause
- **SAT is NP-complete** (Cook, 1971)
- **Horn clause entailment is linear time**

Standard Logical Equivalences

- Commutativity, Associativity
- Double-negation: $\neg\neg\alpha \equiv \alpha$
- Contraposition: $\alpha \Rightarrow \beta \equiv \neg\beta \Rightarrow \neg\alpha$
- Implication: $\alpha \Rightarrow \beta \equiv \neg\alpha \vee \beta$
- De Morgan: $\neg(\alpha \wedge \beta) \equiv \neg\alpha \vee \neg\beta$; $\neg(\alpha \vee \beta) \equiv \neg\alpha \wedge \neg\beta$
- Distributivity: $\alpha \wedge (\beta \vee \gamma) \equiv (\alpha \wedge \beta) \vee (\alpha \wedge \gamma)$

People

Person	Contribution
Boole (1847)	Mathematical Analysis of Logic
Frege (1879)	Begriffsschrift; modern logic
Horn (1951)	Horn form
Davis, Logemann, Loveland (1962)	DPLL
J.A. Robinson (1965)	Resolution
Cook (1971)	SAT is NP-complete
Kautz & Selman (1992)	SATPLAN
Selman et al. (1992)	WALKSAT
Reiter (1991)	Successor-state axioms

Design Paradigms

- Knowledge-based agent: TELL $\rightarrow$ ASK $\rightarrow$ act $\rightarrow$ repeat
- Model checking vs theorem proving vs SAT solving
- CNF as normal form for resolution
- Horn clause specialization $\rightarrow$ efficient forward/backward chaining
- SAT as search: complete (DPLL) vs incomplete (WALKSAT)
- Successor-state axioms solve frame problem
- 1-CNF belief state as conservative approximation

Ch. 8 — First-Order Logic

Named Entities (Terms & Definitions)

Term	Definition
First-Order Logic (FOL)	Assumes world consists of objects with relations among them
Compositionality	Meaning of sentence = function of meaning of parts
Object	A thing in the world
Relation	Link among objects (unary = property, n-ary = relation)
Function	Relation with exactly one value for given input
Ontological commitment	What a language assumes exists. Prop logic: facts. FOL: facts, objects, relations
Epistemological commitment	Possible knowledge states. FOL: true/false/unknown. Probability: [0,1]
Domain	Set of objects in a model (nonempty)
Constant symbol	Stands for an object
Predicate symbol	Stands for a relation
Function symbol	Stands for a function
Arity	Number of arguments a symbol takes
Interpretation	Maps constants → objects, functions → functions, predicates → relations
Term	Expression referring to an object (constant, variable, complex term)
Ground term	Term with no variables
Atomic sentence	Predicate symbol + parenthesized list of terms
Quantifier	$\forall$ (universal) and $\exists$ (existential)
Equality symbol (=)	Two terms refer to same object
Unique-names assumption	Every constant refers to distinct object
Closed-world assumption	Atomic sentences not known true are false
Database semantics	UNA + CWA + domain closure

Ontological vs Epistemological Commitments

Language	Ontological	Epistemological
Propositional logic	facts	true/false/unknown
FOL	facts, objects, relations	true/false/unknown
Temporal logic	facts, objects, relations, times	true/false/unknown
Probability theory	facts	degree of belief $\in [0,1]$
Fuzzy logic	facts with degree of truth	known interval value

FOL Syntax

```

Sentence → AtomicSentence | ComplexSentence
AtomicSentence → Predicate | Predicate(Term, ...) | Term = Term
ComplexSentence → (Sentence) | ¬ Sentence | Sentence ∧ Sentence
                  | Sentence ∨ Sentence | Sentence ⇒ Sentence | Sentence ⇔ Sentence
                  | Quantifier Variable, ... Sentence
Term → Function(Term, ...) | Constant | Variable
Quantifier → ∀ | ∃
Operator Precedence: ¬, =, ∧, ∨, ⇒, ⇔

```

Quantifier Rules

- $\neg \exists x P \equiv \forall x \neg P$; $\neg \forall x P \equiv \exists x \neg P$
- $\forall x P \equiv \neg \exists x \neg P$; $\exists x P \equiv \neg \forall x \neg P$
- $\forall$ is natural with $\Rightarrow$: $\forall x (\text{King}(x) \Rightarrow \text{Person}(x))$
- $\exists$ is natural with $\wedge$: $\exists x (\text{Crown}(x) \wedge \text{OnHead}(x, \text{John}))$
- $\forall x \exists y \text{Loves}(x,y) \neq \exists y \forall x \text{Loves}(x,y)$

Knowledge Engineering Process (7 Steps)

1. Identify the questions
2. Assemble relevant knowledge
3. Decide on vocabulary (ontology)
4. Encode general knowledge (axioms)
5. Encode problem instance
6. Pose queries
7. Debug and evaluate

People

Person	Contribution
Frege (1879)	Begriffsschrift, quantifiers, first FOL
Peirce (1870, 1883)	Logic of relations
Peano (1889)	Notation for FOL
Tarski (1935)	Definition of truth, model theory
McCarthy (1958)	FOL for AI
Robinson (1965)	Resolution

Ch. 9 — Inference in First-Order Logic

Named Entities (Terms & Definitions)

Term	Definition
Universal Instantiation (UI)	From $\forall v \alpha$, infer $\text{SUBST}(\{v/g\}, \alpha)$ for any ground term g
Existential Instantiation (EI)	From $\exists v \alpha$, infer $\text{SUBST}(\{v/k\}, \alpha)$ with new Skolem constant k
Skolem constant	New constant introduced by EI
Generalized Modus Ponens (GMP)	Lifted Modus Ponens with unification
Lifting	Raising inference rules from ground to FOL via unification
Unification	Finding substitution θ making two expressions identical
MGU (Most General Unifier)	Unifier placing fewest restrictions on variables
Standardizing apart	Renaming variables to avoid name clashes
Occur check	Prevents infinite recursion in unification
Resolution	Complete inference for any FOL KB (refutation: prove $\text{KB} \wedge \neg \alpha$ unsatisfiable)
Skolemization	Removing $\exists$ quantifiers by replacing with Skolem functions/constants
Herbrand's theorem	If S unsatisfiable, finite subset of Herbrand base is also
Datalog	FOL definite clauses with no function symbols
Logic programming	Algorithm = Logic + Control (Kowalski)
Prolog	Most widely used logic programming language

Unification Algorithm

```

UNIFY(x, y,  $\theta$ =empty):
  if  $\theta$  = failure: return failure
  if  $x = y$ : return  $\theta$ 
  if VARIABLE?(x): return UNIFY-VAR(x, y,  $\theta$ )
  if VARIABLE?(y): return UNIFY-VAR(y, x,  $\theta$ )
  if COMPOUND?(x) and COMPOUND?(y):
    return UNIFY(ARGS(x), ARGS(y), UNIFY(OP(x), OP(y),  $\theta$ ))
  if LIST?(x) and LIST?(y):
    return UNIFY(REST(x), REST(y), UNIFY(FIRST(x), FIRST(y),  $\theta$ ))
  return failure

```

Forward Chaining (FOL-FC-ASK) Sound, complete for definite clause KBs. For Datalog: terminates in $\leq p \cdot n^k$ iterations. For

general definite clauses with functions: semidecidable.

Backward Chaining (FOL-BC-ASK) Depth-first search → linear space; suffers from repeated states and incompleteness.

Conversion to CNF

1. Eliminate implications
2. Move $\neg$ inwards
3. Standardize variables
4. Skolemize (replace $\exists x P$ with $P(A)$ or $P(F(x_1, \dots, x_n))$)
5. Drop universal quantifiers
6. Distribute $\vee$ over $\wedge$

Resolution Rule $\ell_1 \vee \dots \vee \ell_k, m_1 \vee \dots \vee m_n \rightarrow \text{SUBST}(\theta, \ell_1 \vee \dots \vee \ell_{i-1} \vee \ell_{i+1} \vee \dots \vee \ell_k \vee m_1 \vee \dots \vee m_{j-1} \vee m_{j+1} \vee \dots \vee m_n)$ where $\text{UNIFY}(\ell_i, \neg m_j) = \theta$

Completeness Proof Structure

1. If S unsatisfiable → finite subset of Herbrand base unsatisfiable (Herbrand)
2. Propositional resolution complete for ground (ground resolution theorem)
3. Lifting lemma: ground proof → corresponding FOL proof

Inference Complexity

KB Type	Entailment	Decidable?
Propositional logic	SAT	Decidable (NP-complete)
FOL (general)	Validity	Semidecidable
Datalog	Entailment	Decidable (polynomial)
Definite clauses with functions	Entailment	Semidecidable

People

Person	Contribution
Herbrand (1930)	Herbrand's theorem, unification
Turing (1936) / Church (1936)	Undecidability of FOL validity
Gödel (1930, 1931)	Complete proof procedure for FOL; Incompleteness
Robinson (1965)	Resolution
Kowalski	Algorithm = Logic + Control

Person	Contribution
Colmerauer (1972)	Prolog
Forgy (1982)	Rete algorithm

Ch. 10 — Knowledge Representation

Named Entities (Terms & Definitions)

Term	Definition
Ontological engineering	Creating representations for general concepts
Upper ontology	General framework at top of hierarchy (Anything → AbstractObjects, PhysicalObjects)
Category	Organization of objects into classes
Reification	Turning a proposition into an object
Inheritance	Properties of superclass inherited by subclasses
Taxonomy	Subclass relations organizing categories
Partition	Disjoint + exhaustive decomposition
Event calculus	Formal representation of events, fluents, time points
Fluent	Aspect of the world that changes over time
Modal logic	Logic with modal operators (K knowledge, □ necessity, ◇ possibility)
Possible world	Complete consistent state of affairs
Accessibility relation	Links worlds w.r.t. modal operator
Description logics	Formal language for category definitions; tractable subsumption
Semantic networks	Graphical nodes (objects) and labeled links (relations)
Nonmonotonic logic	Logic where new evidence can retract conclusions
Circumscription	Assume predicates false except where known true
Default logic	$P:J_1, \dots, J_n / C$ (if P true and J_i consistent, conclude C)
Truth maintenance system (TMS)	Handles retraction of inferences

Term	Definition
Belief revision	Retracting incorrect inferences with new information

Upper Ontology Hierarchy

Anything

├ AbstractObjects (Sets, Numbers, RepresentationalObjects, Intervals, Places, Pr

└ GeneralizedEvents

Time Interval Relations (Allen, 1983) Meet(i,j), Before(i,j), After(j,i), During(i,j), Overlap(i,j), Starts(i,j), Finishes(i,j), Equals(i,j)

People

Person	Contribution
Aristotle	Categories, genus/species
Peirce (1909)	Existential graphs
Quillian (1961)	First AI semantic networks
Minsky (1975)	Frames
Hayes (1979, 1985)	Naive Physics
Kowalski & Sergot (1986)	Event calculus
Allen (1983, 1984)	Time intervals
McCarthy (1980)	Circumscription
Reiter (1980)	Default logic
Doyle (1979)	TMS

Ch. 11 — Automated Planning

Named Entities (Terms & Definitions)

Term	Definition
Classical planning	Finding action sequence to accomplish goal in discrete, deterministic, static, fully observable environment
PDDL	Planning Domain Definition Language
Action schema	Template with name, variables, precondition, effect
Precondition	Conjunction of literals that must be true

Term	Definition
Effect	Conjunction of literals (positive/negative)
Applicable action	$s \models \text{Precond}(a)$
RESULT(s, a)	$(s - \text{DEL}(a)) \cup \text{ADD}(a)$
Regression search	Backward search from goal to initial state
SATPLAN	Translate PDDL to propositional SAT
Planning graph	Encodes constraints on actions, preconditions, effects, mutual exclusions
Partial-order planning	Plan as graph with ordering constraints
HTN planning	Hierarchical Task Network with high-level actions and refinements
Downward refinement	Every high-level plan claiming goal has implementation that does
property	
Angelic semantics	Agent chooses implementation; if reachable set n goal $\neq \emptyset$, plan works
Conformant planning	Sensorless planning
Contingent planning	Conditional branching based on percepts
Execution monitoring	Determining when replanning needed
Scheduling	Adding temporal info; critical path method
Job-shop scheduling	Jobs with ordered actions, durations, resources

PDDL Action Schema

Action(ActionName(vars), PRECOND: conjunction, EFFECT: conjunction)

Planning Heuristics

Heuristic	Admissible?	Issues
Ignore-preconditions	Usually not	Too optimistic
Ignore-delete-lists	Usually not	NP-hard optimal
Set cover	No	NP-hard exact
Subgoal independence max	Admissible	May be too low

People

Person	Contribution
Fikes & Nilsson (1971)	STRIPS, Shakey
Sacerdoti (1974, 1977)	ABSTRIPS, NOAH
Blum & Furst (1997)	Graphplan
Kautz & Selman (1998)	SATPLAN, BLACKBOX
Hoffmann (2001, 2005)	FF (Fast Forward) planner
McDermott (1996)	PDDL

Ch. 12 — Quantifying Uncertainty

Named Entities (Terms & Definitions)

Term	Definition
Uncertainty	From partial observability, nondeterminism, or adversaries
Belief state	Set of all possible world states agent might be in
Qualification problem	Cannot deduce plan success; unlimited qualifiers
Degree of belief	Numerical measure [0,1] of confidence
Probability theory	Ontological commitment same as logic; epistemological commitment allows degrees
Decision theory	Probability theory + Utility theory
MEU (Maximum Expected Utility)	Rational agent chooses action maximizing expected utility
Sample space (Ω)	Set of all possible worlds; mutually exclusive and exhaustive
Event	Set of possible worlds (proposition)
Prior probability	Belief without any other information
Posterior probability	Belief given evidence
Random variable	Function from Ω to some range
Bernoulli distribution	Boolean random variable
Joint probability distribution	Distribution on multiple variables
Full joint distribution	Joint over all random variables
Kolmogorov's axioms	$0 \leq P(\omega) \leq 1$, $\sum P(\omega)=1$, inclusion-exclusion

Term	Definition
Marginalization	Summing out other variables
Bayes' rule	$P(b)$
Conditional independence	$P(X,Y)$
Naive Bayes	$P(\text{Cause}, \text{Effect}_1, \dots, \text{Effect}_n) = P(\text{Cause}) \prod_i P(\text{Effect}_i)$
Frequentist / Objectivist / Subjectivist	Three interpretations of probability

Processes / Algorithms Decision-Theoretic Agent: Maintain belief state → update based on action/percept → calculate outcome probabilities → select action maximizing expected utility.

Probabilistic Inference (full joint distribution): $P(X|e) = \alpha \sum_y P(X,e,y)$. Complexity $O(2^n)$.

Bayes' Rule with normalization: $P(Y|X) = \alpha P(X|Y) P(Y)$

Formulas & Equations

- **Conditional probability:** $P(a|b) = P(a \wedge b) / P(b)$ for $P(b) > 0$
- **Product rule:** $P(a \wedge b) = P(a|b)P(b)$
- **Inclusion-exclusion:** $P(a \vee b) = P(a) + P(b) - P(a \wedge b)$
- **Marginalization:** $P(Y) = \sum_z P(Y, Z=z)$
- **Conditioning:** $P(Y) = \sum_z P(Y|z)P(z)$
- **Bayes' rule:** $P(b|a) = P(a|b)P(b)/P(a)$
- **Bayes with evidence:** $P(Y|X,e) = P(X|Y,e)P(Y|e)/P(X|e)$
- **Naive Bayes:** $P(\text{Cause}|e) = \alpha P(\text{Cause}) \prod_i P(e_i|\text{Cause})$

Kolmogorov's Axioms

- (1) $0 \leq P(\omega) \leq 1$; (2) $\sum_{\omega \in \Omega} P(\omega) = 1$; (3) $P(a \vee b) = P(a) + P(b) - P(a \wedge b)$

Dutch book argument (de Finetti): violating axioms guarantees loss.

People

Person	Contribution
Thomas Bayes (1702-1761)	Bayes' rule
Laplace	General Bayes rule, principle of indifference
Kolmogorov (1933)	Axiomatic probability

Person	Contribution
de Finetti	Dutch book argument
Von Neumann & Morgenstern	Utility theory

Ch. 13 — Probabilistic Reasoning

Named Entities (Terms & Definitions)

Term	Definition
Bayesian network	DAG with CPTs; compact representation of joint distribution
CPT	Conditional probability table for each node given parents
Topological ordering	Parents before children
Locally structured system	Each subcomponent interacts with bounded number of others
Markov blanket	Parents, children, children's parents; d-separates node from all others
D-separation	Determines conditional independence in Bayes net
Noisy-OR	Generalization of logical OR with inhibition probabilities
Linear-Gaussian	Child Gaussian with mean linear in parent values
Variable elimination	Exact inference summing out variables from pointwise products of factors
Factor	Matrix indexed by argument variable values
Pointwise product	$f \times g = h$ with union of variables
Rejection sampling	Sample from prior, reject inconsistent with evidence
Likelihood weighting	Fix evidence, sample others, weight = product of evidence likelihoods
Gibbs sampling	MCMC; sample each nonevidence variable given Markov blanket
Metropolis-Hastings	General MCMC; propose from $q(x')$
Stationary distribution $\pi(x)$	Distribution unchanged under transition kernel
Detailed balance	$\pi(x)k(x \rightarrow x') = \pi(x')k(x' \rightarrow x)$
Do-calculus	Notation $\text{do}(X_j = x_{jk})$ for intervening

Term	Definition
Adjustment formula	$P(X_i$
Back-door criterion	Set Z closing all back-door paths

Processes / Algorithms Constructing a Bayes Net: 1. Determine variables; order causes before effects 2. For each X_i , choose minimal $\text{Parents}(X_i)$ s.t. $P(X_i|X_{i-1}, \dots, X_1) = P(X_i|\text{Parents}(X_i))$ 3. Write CPTs

Variable Elimination: - For each V in order: make factor from V's CPT; if V hidden, sum out V - Return normalized pointwise product of remaining factors - Complexity: linear for polytrees; exponential for multiply connected

Gibbs Sampling: - Initialize x with random nonevidence values - For each step: choose Z_i , sample from $P(Z_i|\text{mb}(Z_i))$ - Stationary distribution = true posterior

Formulas & Equations

- **Bayes net joint:** $P(x_1, \dots, x_n) = \prod_{i=1}^n P(x_i|\text{parents}(X_i))$
- **Chain rule:** $P(x_1, \dots, x_n) = \prod P(x_i|x_{i-1}, \dots, x_1)$
- **Noisy-OR:** $P(x_i|\text{parents}) = 1 - \prod_{j: X_j=\text{true}} q_j$
- **Gibbs blanket:** $P(x_i|\text{mb}(X_i)) = \alpha P(x_i|\text{parents}(X_i)) \prod_{Y_j \in \text{Children}(X_i)} P(y_j|\text{parents}(Y_j))$
- **Adjustment formula:** $P(X_i=x_i|\text{do}(X_j=x_{jk})) = \frac{\sum_{\text{parents}(X_i)} P(x_i|x_{jk}, \text{parents}) P(\text{parents})}{\sum_{\text{parents}(X_i)} P(\text{parents})}$

People

Person	Contribution
Judea Pearl (1982-2000)	Bayes nets, d-separation, do-calculus
Geman & Geman (1984)	Gibbs sampler
Metropolis et al. (1953)	MCMC origin
Hastings (1970)	Metropolis-Hastings

Ch. 14 — Probabilistic Reasoning over Time

Named Entities (Terms & Definitions)

Term	Definition
Markov assumption	Current state depends on finite fixed number of previous states
First-order Markov process	$P(X_t)$
Time-homogeneous Transition model	Transition model same for all t
Sensor model	$P(E_t)$
Filtering (state estimation)	$P(X_t)$
Prediction	$P(X_{t+k})$
Smoothing	$P(X_k)$
Most likely explanation	$\text{argmax } P(x_{1:t})$
Forward message $f_{1:t}$	$P(X_t)$
Backward message $b_{k+1:t}$	$P(e_{k+1:t})$
HMM	Single discrete state variable
Kalman filter	Continuous state, linear-Gaussian
DBN	Bayes net with replicated time slices
Particle filtering	SIS + resampling

Processes / Algorithms Filtering recursion: $P(X_{t+1}|e_{1:t+1}) = \alpha P(e_{t+1}|X_{t+1}) \sum_{x_t} P(X_{t+1}|x_t) P(x_t|e_{1:t})$

Forward-Backward Algorithm (smoothing): 1. Forward pass: compute $fv[0..t]$ 2. Backward pass: compute $sv[i] = \text{normalize}(fv[i] \times b)$

Viterbi (most likely sequence): $m_{1:t+1} = P(e_{t+1}|X_{t+1}) \max_{x_t} \{x_t\} [P(X_{t+1}|x_t) \times \max_{x_{1:t-1}} \{x_{1:t-1}\} P(x_{1:t-1}, x_t, e_{1:t})]$ Sum $\rightarrow$ max; no normalization; record best predecessor.

Kalman Filter: - Prediction: $\mu_{t+1|t} = F\mu_t$; $\Sigma_{t+1|t} = F\Sigma_t F^T + \Sigma_x$ - Update: $K_{t+1} = \Sigma_{t+1|t} H^T (H\Sigma_{t+1|t} H^T + \Sigma_z)^{-1}$ - $\mu_{t+1} = \mu_{t+1|t} + K_{t+1}(z_{t+1} - H\mu_{t+1|t})$ - $\Sigma_{t+1} = (I - K_{t+1}H)\Sigma_{t+1|t}$

Particle Filtering (each time step): 1. Propagate each sample from $P(X_{t+1}|X_t)$ 2. Weight each sample by $P(e_{t+1}|X_{t+1})$ 3. Resample N particles with replacement by weight

Formulas & Equations

- **Joint:** $P(X_{0:t}, E_{1:t}) = P(X_0) \prod_i P(X_i|X_{i-1}) P(E_i|X_i)$
- **Filtering:** $P(X_{t+1}|e_{1:t+1}) = \alpha P(e_{t+1}|X_{t+1}) \sum P(X_{t+1}|x_t) P(x_t|e_{1:t})$

- **Smoothing:** $P(X_k|e_{1:t}) = \alpha f_{1:k} \times b_{k+1:t}$
- **HMM forward:** $f_{1:t+1} = \alpha O_{t+1} T^T f_{1:t}$
- **Kalman model:** $P(x_{t+1}|x_t) = N(x_{t+1}; Fx_t, \Sigma_x); P(z_t|x_t) = N(z_t; Hx_t, \Sigma_z)$

People

Person	Contribution
Andrei Markov (1856-1922)	Markov processes
Rudolf Kalman (1960)	Kalman filter
Andrew Viterbi (1967)	Viterbi algorithm
Baum & Petrie (1966)	HMM, forward-backward
Gordon, Salmond, Smith (1993)	Particle filtering

Ch. 15 — Probabilistic Programming

Named Entities (Terms & Definitions)

Term	Definition
Probabilistic Programming Language (PPL)	Defines distributions over execution traces via stochastic elements
Relational Probability Model (RPM)	Uses database semantics for finite possible worlds
Open-Universe Probability Model (OUPM)	Full FOL semantics; object existence/identity uncertainty
Database semantics	UNA + domain closure
Basic random variable	Instantiated function with each object combination
Grounding / Unrolling	Constructing equivalent BN from RPM
Number statement	Conditional distributions over numbers of objects
Data association problem	Associating observations with generating objects
Rao-Blackwellization	Exact inference for subset conditioned on sampled values
Generative program	Executable code where random choices define random variables

Term	Definition
Markov Logic Network (MLN)	Maximum-entropy probabilistic logic
Probability logic	$P(\phi) \geq p$ constrains distribution over possible worlds

Probability Model Hierarchy

- **Atomic:** HMMs
- **Factored:** Bayesian networks, DBNs, Kalman filters
- **Structured:** RPMs, OUPMs, PPLs

Empirical Evidence

- Citation matching: OUPM error rate 2-3× lower than CiteSeer
- NET-VISA: missed 11.1% vs UN SEL3's 27.4% for magnitude 3-4 events
- TrueSkill: serves hundreds of millions of users daily

Ch. 16 — Making Simple Decisions

Named Entities (Terms & Definitions)

Term	Definition
Decision theory	Probability theory + Utility theory
Utility function $U(s)$	Number expressing desirability of a state
Expected utility $EU(a)$	$\sum_{s'} P(\text{RESULT}(a)=s') U(s')$
MEU principle	Choose action maximizing expected utility
Lottery	$[p_1, S_1; p_2, S_2; \dots; p_n, S_n]$
Risk-averse / Risk-seeking / Risk-neutral	Concave / convex / linear utility curve
Certainty equivalent	Value agent accepts in lieu of lottery
Insurance premium	EMV - certainty equivalent
Optimizer's curse	Estimated EU of best choice too high due to selection bias
Certainty effect	Attraction to certain gains (Kahneman & Tversky)

Term	Definition
Multiattribute utility theory	Comparing outcomes with multiple attributes
Stochastic dominance	A_1 stochastically dominates A_2 if $\forall x \int p_1 \leq \int p_2$
Preference independence / MPI	Each attribute independent in tradeoffs
Decision network	BN + decision nodes + utility nodes
VPI (Value of Perfect Information)	Expected improvement from learning exact value

Rational Preference Axioms (von Neumann-Morgenstern)

1. Orderability, 2. Transitivity, 3. Continuity, 4. Substitutability, 5. Monotonicity, 6. Decomposability

Formulas & Equations

- **EU:** $EU(a) = \sum \{s'\} P(\text{RESULT}(a)=s') U(s')$
- **MEU:** $\text{action} = \text{argmax}_a EU(a)$
- **Positive affine transformation:** $U'(S) = a \cdot U(S) + b, a > 0$
- **Lottery utility:** $U([p_1, S_1; \dots; p_n, S_n]) = \sum p_i U(S_i)$
- **Additive value function:** $V(x_1, \dots, x_n) = \sum V_i(x_i)$
- **VPI:** $VPI(E_j) = [\sum P(e_j) \cdot EU(\alpha_{\{e_j\}}|e_j)] - EU(\alpha)$
- **VPI properties:** Nonnegative, not additive, order-independent

Rules, Laws & Theorems

- Existence of utility function from axioms
- VPI nonnegativity theorem
- Deference theorem: $EU(\text{defer}) \geq EU(a)$ if uncertainty about human utility
- Revelation principle: any mechanism $\rightarrow$ equivalent truth-revealing
- Revenue equivalence theorem

People

Person	Contribution
von Neumann & Morgenstern (1944)	Utility theory
Kahneman & Tversky	Prospect theory, cognitive biases

Ch. 17 — Making Complex Decisions

Named Entities (Terms & Definitions)

Term	Definition
MDP	$\langle S, A(s), P(s') \rangle$
Policy π	Action for each state
Optimal policy π^*	Policy yielding highest expected utility
Bellman equation	$U(s) = \max_a \sum P(s') R(s, a, s')$
Q-function	$Q(s, a) = \sum P(s') R(s, a, s')$
Value iteration	Bellman update; contraction by $\gamma \rightarrow$ converges
Policy iteration	Policy evaluation + policy improvement
Shaping theorem	$R'(s, a, s') = R(s, a, s') + \gamma \Phi(s') - \Phi(s)$ leaves optimal policy unchanged
N-armed bandit	n arms with unknown reward distributions
Gittins index	Value λ for arm; optimal = pull highest index
UCB	$\hat{\mu}_i + g(N)/\sqrt{N_i}$
Thompson sampling	Choose arm by probability it's optimal
POMDP	MDP + sensor model $P(e)$
Belief state b	Distribution over possible states
POMCP	Particle filtering + UCT for POMDPs

Processes / Algorithms Value Iteration:

```

repeat:
     $U \leftarrow U'$ ;  $\delta \leftarrow 0$ 
    for each  $s$ :  $U'[s] \leftarrow \max_a \sum P(s'|s, a) [R(s, a, s') + \gamma U[s']]$ 
until  $\delta \leq \epsilon(1-\gamma)/\gamma$ 

```

Convergence: Bellman update contracts by γ .

Policy Iteration:

```

repeat:
     $U \leftarrow \text{POLICY-EVALUATION}(\pi, U, \text{mdp})$ 
    for each  $s$ :  $\pi[s] \leftarrow a^* = \arg\max_a Q\text{-VALUE}(\text{mdp}, s, a, U)$ 
until  $\pi$  unchanged

```

POMDP belief update: $b'(s') = \alpha P(e|s') \sum_s P(s'|s, a) b(s)$

Formulas & Equations

- **Bellman:** $U(s) = \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma U(s')]$
- **Q-function:** $Q(s, a) = \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma \max_{a'} Q(s', a')]$

- **Infinite horizon bound:** $\sum \gamma^t R \leq R_{\max}/(1-\gamma)$
- **Shaping:** $R'(s,a,s') = R(s,a,s') + \gamma\Phi(s') - \Phi(s)$
- **Gittins index:** $\lambda = \max_{\{T>0\}} E(\sum \gamma^t R_t)/E(\sum \gamma^t)$
- **POMDP belief:** $b'(s') = \alpha P(e|s') \sum_s P(s'|s,a) b(s)$

People

Person	Contribution
Bellman (1957)	Dynamic programming, Bellman equation
Gittins (1974, 1989)	Gittins index
Sondik (1971)	First POMDP value iteration
Kocsis & Szepesvári (2006)	UCT
Silver & Veness (2011)	POMCP

Ch. 18 — Multiagent Decision Making

Named Entities (Terms & Definitions)

Term	Definition
Multiagent system	Environment with multiple decision-making actors
Game theory	Theory of strategic decision making
Normal form game	Players, actions, payoff function
Payoff matrix	Combined matrix for two-player games
Strategy	Policy in game theory
Pure strategy	Deterministic policy
Mixed strategy	Randomized policy
Prisoner's dilemma	Dominant strategy leads to worse outcome for both
Dominant strategy	Better regardless of others' choices
Nash equilibrium	No player can unilaterally improve payoff
Pareto optimality	No player better off without making another worse off
Zero-sum game	Payoffs sum to zero
Repeated game	Multiple rounds of stage game
Subgame perfect Nash equilibrium	Nash in every subgame
Cooperative game	Binding agreements possible
Shapley value	Average marginal contribution over all orderings
Core	Set of imputations where no coalition can do better alone
Mechanism design	Define rules so collective good is maximized
Vickrey auction	Sealed-bid second-price; truth-revealing
VCG mechanism	Truth-revealing, maximizes global utility
Arrow's theorem	No perfect social welfare function

Term	Definition
Assistance game	Two-person game with shared payoff

Formulas & Equations

- **Shapley value:** $\phi_i(G) = (1/n!) \sum_{p \in P} mc_i(p_i)$
- **Marginal contribution:** $mc_i(C) = v(C \cup \{i\}) - v(C)$
- **Core:** $\sum_i x_i = v(N)$; $\sum_{i \in C} x_i \geq v(C)$; $x_i \geq v(\{i\})$
- **Zeuthen risk:** $risk_i = (U_i(\text{conceding}) - U_i(\text{conflict})) / (U_i(\text{not conceding}) - U_i(\text{conflict}))$

Rules, Laws & Theorems

- **Nash's theorem (1950):** Every game has at least one Nash equilibrium in mixed strategies
- **von Neumann's theorem (1928):** Every two-player zero-sum game has maximin equilibrium
- **Arrow's theorem:** No social welfare function for ≥ 3 outcomes satisfies all desired conditions
- **Gibbard-Satterthwaite theorem:** Any reasonable social choice function with > 2 outcomes is either manipulable or a dictatorship
- **Revelation principle:** Any mechanism $\rightarrow$ equivalent truth-revealing mechanism
- **Revenue equivalence theorem:** Private-value auctions yield same expected revenue

People

Person	Contribution
von Neumann (1928, 1944)	Maximin equilibrium, game theory
Nash (1950)	Nash equilibrium
Arrow (1951)	Arrow's impossibility theorem
Shapley (1953a)	Shapley value
Selten	Subgame perfect equilibrium
Vickrey (1961)	Vickrey auction

Ch. 19 — Learning from Examples

Named Entities (Terms & Definitions)

Term	Definition
Supervised learning	Learning from labeled examples
Unsupervised learning	Finding patterns in unlabeled data
Reinforcement learning	Learning from rewards
Decision tree learning	Entropy, information gain, ID3 algorithm
Model selection	Cross-validation, overfitting, pruning
PAC learning	Probably Approximately Correct learning
VC dimension	Measure of model capacity
Bias-variance tradeoff	Error = bias ² + variance + noise
Linear regression	MSE, gradient descent, normal equation
Logistic regression	Classification with sigmoid
SVM	Max-margin classifier with kernel trick
k-NN	Nonparametric, nearest neighbors
Ensemble learning	Bagging, random forests, boosting (AdaBoost), stacking
Developing ML systems	Data pipelines, feature engineering, error analysis

Ch. 20 — Learning Probabilistic Models

Named Entities (Terms & Definitions)

Term	Definition
Maximum likelihood estimation	Choose parameters maximizing data likelihood
Bayesian parameter estimation	Posterior over parameters
Learning Bayesian networks	Complete data: parameter + structure learning
EM algorithm	E-step (expectation), M-step (maximization)
Gaussian mixture models	Weighted sum of Gaussian components

Ch. 21 — Deep Learning

Named Entities (Terms & Definitions)

Term	Definition
Feedforward network	Perceptron, multilayer, activation functions (ReLU, sigmoid, tanh)
Computation graphs	Forward pass, backward pass (backpropagation)
Convolutional networks (CNN)	Convolution, pooling, stride, padding; LeNet, AlexNet, VGG, ResNet
SGD	Stochastic gradient descent
Momentum	Accelerates SGD
Adam	Adaptive moment estimation
Batch normalization	Normalizes layer inputs
Dropout	Randomly drops units during training
Data augmentation	Increases training data variety
Regularization	L1/L2, early stopping
RNN	Recurrent neural network
LSTM	Long Short-Term Memory with gating
GRU	Gated recurrent unit
Autoencoders	Unsupervised representation learning
GANs	Generative adversarial networks
Transfer learning	Pretraining + fine-tuning

Learning Algorithms

- Backpropagation: forward pass → loss → backward pass → gradient updates
- CNN: convolution layers extract features, pooling reduces dimensionality, fully connected classifies
- RNN: shared weights across time, backprop through time
- LSTM: forget gate, input gate, output gate control information flow

Ch. 22 — Reinforcement Learning

Named Entities (Terms & Definitions)

Term	Definition
Reward hypothesis	Goals = maximization of expected cumulative reward
Return	Sum of discounted rewards
Discounting	$\gamma \in [0,1]$
Passive RL	Utility estimation from fixed policy
Direct utility estimation	Average observed returns
Adaptive dynamic programming (ADP)	Learn transition model, solve MDP

Term	Definition
Temporal-difference (TD) learning	Update using next estimate
Active RL	Agent selects actions
Q-learning	Model-free TD learning
Exploration vs exploitation	ϵ -greedy, softmax
DQN	Deep Q-network (function approximation)
Policy search	Policy gradients, REINFORCE
Inverse RL	Learning reward from demonstrations
Apprenticeship learning	Learning from expert behavior
MaxEnt IRL	Maximum entropy inverse RL

Applications TD-Gammon, AlphaGo, AlphaZero, Dota 2, robotics, self-driving

Ch. 23 — Natural Language Processing

Named Entities (Terms & Definitions)

Term	Definition
Language model	Probability distribution over strings
Bag-of-words	Naive Bayes on words; assumes independence
N-gram model	Markov chain; word depends on n-1 previous
Tokenization	Dividing text into words
Smoothing	Reserving probability for unseen n-grams
Backoff model	Using (n-1)-grams when n-gram count low
Part of speech (POS)	Lexical category (noun, verb, etc.)
Penn Treebank	3M+ words annotated with POS and parse trees
Probabilistic CFG (PCFG)	Grammar with probabilities
CYK algorithm	$O(n^3m)$ bottom-up chart parsing
Compositional semantics	Meaning = f(meaning of subphrases)
λ-calculus	Notation for predicates
Lexical / Syntactic / Semantic ambiguity	Three types of ambiguity
Dependency grammar	Binary relations between lexical items

Processes / Algorithms **N-gram probability:** $P(w_j|w_{1:j-1}) = P(w_j|w_{j-n+1:j-1})$ **Naive Bayes for text:** $P(\text{Class}|w_{1:N}) = \alpha P(\text{Class}) \prod_j P(w_j|\text{Class})$ **POS with HMM:** Viterbi on transition $P(C_t|C_{t-1})$ and sensor $P(W_t|C_t)$ **CYK:** Lexical insertion $\rightarrow$ syntactic combination of shortest spans first

Formulas & Equations

- **N-gram joint:** $P(w_1:N) = \prod_j P(w_j | w_{j-n+1:j-1})$
- **Linear interpolation:** $\hat{P}(C_i | C_{i-2:i-1}) = \lambda_3 P(C_i | C_{i-2:i-1}) + \lambda_2 P(C_i | C_{i-1}) + \lambda_1 P(C_i)$
- **CYK:** $P[X, i, k] = \max_j P[Y, i, j] \times P[Z, j+1, k] \times p$
- **β -reduction:** $(\lambda x \text{ Loves}(x, Bo))(Ali) = \text{Loves}(Ali, Bo)$

People

Person	Contribution
Markov (1913)	N-gram letter models
Shannon (1949)	First n-gram word models
Chomsky (1956, 1957)	Context-free grammars
Laplace (1816)	Add-one smoothing

Ch. 24 — Deep Learning for Natural Language Processing

Named Entities (Terms & Definitions)

Term	Definition
Word embedding	Low-dimensional dense vector for a word
GloVe	Word embeddings from co-occurrence matrix factorization
Word2Vec	Neural word embedding model
RNN	Processes sequences with shared weights
Bidirectional RNN	Left-to-right + right-to-left
LSTM	Gating to control information flow
Seq2seq	Encoder-decoder RNN for translation
Attention mechanism	Context-based source summarization per target step
Transformer	Self-attention architecture
Self-attention	Query-key-value within sequence
Multiheaded attention	Multiple parallel attention mechanisms
Masked language model (MLM)	Bidirectional prediction of masked words
BERT	Bidirectional Encoder Representations from Transformers
GPT-2	Generative pretrained transformer (1.5B parameters)
T5	Text-to-Text Transfer Transformer
Perplexity	2^H where H = entropy

Processes / Algorithms **Self-attention:** $q_i = W_q x_i$, $k_i = W_k x_i$,
 $v_i = W_v x_i$; $a_{ij} = \text{softmax}((q_i \cdot k_j) / \sqrt{d})$; $c_i = \sum a_{ij} v_j$

GloVe: $E_i \cdot E'_k = \log(P_{ij})$ where $P_{ij} = X_{ij}/X_i$

RNN LM: $z_t = f(W_{xz} x_t + W_{zz} z_{t-1})$; $y_t = \text{softmax}(W_{zy} z_t)$

Comparisons

Model	Context	Parameters	Parallelism
N-gram	Fixed n	$O(v^n)$	No
RNN	Variable	$O(1)$	No
LSTM	Variable	$O(1)$	No
Transformer	Variable	$O(1)$	Yes (full parallel)

People

Person	Contribution
Bengio et al. (2003)	Neural network LMs
Mikolov et al. (2013)	Word2Vec
Vaswani et al. (2018)	Transformer
Devlin et al. (2018)	BERT

Ch. 25 — Computer Vision

Named Entities (Terms & Definitions)

Term	Definition
Passive sensing	No signal emitted
Active sensing	Emitting signal (radar, lidar)
Pinhole camera	Simple camera without lens
Scaled orthographic projection	Depth variation « distance
Edge detection	Boundaries via intensity discontinuities
Segmentation	Partitioning image into regions
CNN	Convolutional neural network
Image classification	Category label for entire image
Object detection	Locate and classify objects
Semantic segmentation	Label each pixel with object class
Binocular stereopsis	Depth from disparity
Optical flow	Perceived motion of brightness patterns
Structure from motion	3D from moving camera

Processes / Algorithms **Pinhole projection:** $(x,y) = (fX/Z, fY/Z)$
Edge detection (Canny): smooth $\rightarrow$ gradient $\rightarrow$ non-max suppression $\rightarrow$ hysteresis thresholding **CNN:** convolution $\rightarrow$ pooling $\rightarrow$ fully connected; trained with backprop **Stereopsis:** depth = $f \cdot b/d$ where d = disparity **Convolution:** $(I * K)[i,j] = \sum_m \sum_n I[i+m, j+n] K[m,n]$

People

Person	Contribution
Krizhevsky et al. (2012)	AlexNet
LeCun et al. (1998)	LeNet
Canny (1986)	Edge detection

Ch. 26 — Robotics

Named Entities (Terms & Definitions)

Term	Definition
Robot	Physical agent manipulating the physical world
Effector	Device asserting physical forces
Sensor	Device perceiving environment
Configuration space (C-space)	Complete robot pose specification
Motion planning	Finding path in C-space
PID controller	Proportional-Integral-Derivative feedback
SLAM	Simultaneous Localization and Mapping
Probabilistic roadmap (PRM)	Sampling-based motion planning
RRT	Rapidly-exploring Random Tree
Model predictive control (MPC)	Online replanning with rolling horizon
LQR	Linear Quadratic Regulator

Processes / Algorithms **PID controller:** $u(t) = K_p e(t) + K_i \int e(\tau) d\tau + K_d de(t)/dt$ **PRM:** Sample milestones $\rightarrow$ connect nearby $\rightarrow$ search graph **RRT:** Start tree $\rightarrow$ sample random config $\rightarrow$ extend toward sample $\rightarrow$ check collisions **Kalman filter update:** Predict $(\hat{x}_t|_{t-1}, P_t|_{t-1}) \rightarrow$ Update $(K_t, \hat{x}_t|_t, P_t|_t)$

People

Person	Contribution
Brooks (1986)	Subsumption architecture
Thrun et al. (2005)	Probabilistic robotics

Ch. 27 — Philosophy, Ethics, and Safety of AI

Named Entities (Terms & Definitions)

Term	Definition
Weak AI	Machines behave intelligently
Strong AI	Machines have actual minds
Turing test	Behavioral test for intelligence
Chinese room (Searle)	Syntax $\neq$ semantics
Consciousness	Subjective experience (qualia)
Dualism	Mind and body separate
Physicalism	Mind from physical processes
Lethal autonomous weapons (LAWs)	Autonomous target selection and engagement
Differential privacy	ϵ -privacy via noise addition
k-anonymity	Each record indistinguishable from k-1 others
Federated learning	Training without centralizing data
Societal bias	Prejudice in data perpetuated by ML
Fairness through unawareness	Ignore protected attributes
Equal opportunity	Same correct classification rate
Demographic parity	Same outcomes across groups
Value alignment problem	System's goal matches human intent
Reward hacking	Gaming the reward function
Negative side effects	Unintended consequences
Superintelligence	AI vastly exceeding human ability
Technological singularity	Recursive self-improvement explosion
Technological unemployment	Job loss from automation
Asimov's laws of robotics	0-3 laws for robot ethics
Explainable AI (XAI)	Systems that explain decisions

AI Safety Approach (Russell)

1. Machine's only objective: maximize human preferences
2. Machine initially uncertain about preferences
3. Human behavior provides evidence about preferences

Fairness Criteria

Criterion	Focus	Problem
Individual fairness	Similar individuals	Hard to define "similar"
Group fairness	Class statistics	Ignores individuals
Fairness through unawareness	Delete protected attributes	Latent variable prediction
Equal outcome / demographic parity	Same rates	May sacrifice accuracy
Equal opportunity	Same correct-classification rate	Ignores bias in training data

Differential Privacy $|\log P(Q(D)=y) - \log P(Q(D+r)=y)| \leq \epsilon$ — add calibrated noise to queries.

People

Person	Contribution
Turing (1950)	Computing Machinery and Intelligence
Searle (1980)	Chinese room argument
Asimov (1942)	Three Laws of Robotics
Weizenbaum (1976)	Computer Power and Human Reason
Russell (2019)	Human Compatible
Bostrom (2014)	Superintelligence

Ch. 28 — The Future of AI

Named Entities (Terms & Definitions)

Term	Definition
MEMS	Micro-electromechanical systems
Preference uncertainty	Agent uncertain about human objectives
Inverse RL	Learning reward from expert
LTL	Linear temporal logic
Differentiable programming	End-to-end differentiable systems
Predictive learning	Unsupervised world modeling
Shared model	Pretrained model as starting point
Anytime algorithm	Interruptible; quality improves over time
Metareasoning	Optimal selection of computations
Bounded optimality	Best program for given architecture
General AI (HLAI)	Human-level across diverse tasks

Future AI Components

1. Better sensors/actuators (cheaper lidar, MEMS)
2. State representation (logic + probability + neural)
3. Action selection (hierarchical planning, HRL)
4. Preference specification (IRL, LTL)
5. Learning (weakly supervised, predictive, transfer, differentiable)

Formulas & Equations

- agent = architecture + program
- Bounded optimality: for fixed architecture, $\exists$ program achieving best possible performance
- Ultimate computing limit: 1kg device $\sim 10^{51}$ ops/sec

People

Person	Contribution
Lloyd (2000)	Ultimate physical limits of computation
Russell & Wefald (1989)	Metareasoning
Russell & Subramanian (1995)	Bounded optimality

Cross-Cutting Topics

Design Paradigms & Meta-Methods

- **Problem relaxation** $\rightarrow$ admissible heuristics (supergraph property)

- **Divide and conquer** — pattern databases, DP
- **Dynamic programming** — Bellman equations, variable elimination, forward-backward, Viterbi
- **Greedy algorithms** — hill climbing, greedy best-first search
- **Backtracking search** — CSP backtracking, DPLL
- **Branch and bound** — A*, alpha-beta pruning
- **Conflict-directed backjumping** — CSPs
- **AND-OR search / Conditional planning** — nondeterministic environments
- **Expectiminimax / Minimax** — adversarial search
- **Monte Carlo methods** — MCTS, rejection sampling, importance sampling, MCMC
- **Bayesian inference** — Bayes nets, filtering, variable elimination
- **Deep learning** — end-to-end representation learning, CNNs, RNNs, transformers
- **Reinforcement learning** — model-free (Q-learning) vs model-based (ADP)
- **Transfer learning & pretraining** — BERT, GPT-2, ImageNet pretraining
- **Ensemble methods** — bagging, boosting, random forests
- **Attention mechanism** — transformer self-attention, seq2seq attention
- **Convolutional processing** — CNNs for vision, text
- **Recurrent processing** — RNNs, LSTMs for sequences
- **Local search** — hill climbing, simulated annealing, genetic algorithms
- **Constraint propagation** — AC-3, forward checking, MAC
- **Satisficing** — accepting good-enough solutions
- **Metareasoning** — reasoning about which computations are worth doing
- **Belief-state search** — transforming partial observability to full observability
- **Coercion** — forcing world to goal without sensing

Proof & Argument Patterns

- **Mathematical induction** — PAC learning bounds, convergence proofs
- **Contradiction** — A* optimality proof: assume suboptimal solution $\rightarrow$ contradiction via $f(n)$ bounds
- **Contraction mapping** — Bellman update contracts by $\gamma \rightarrow$ value iteration converges
- **Reduction** — 3-SAT to Bayes net inference (NP-hardness), CSP to SAT

- **Resolution refutation completeness** — ground resolution theorem + lifting lemma
- **Bellman optimality** — Bellman equation defines optimal policy via DP
- **Nash equilibrium existence** — Nash's theorem via fixed point
- **Dutch book argument** — violating probability axioms guarantees loss
- **Detailed balance → stationarity** — MCMC convergence
- **Refutation** — $\alpha \models \beta$ iff $(\alpha \wedge \neg\beta)$ unsatisfiable
- **Herbrand's theorem** → lifting lemma → FOL resolution completeness
- **Deduction theorem** — $\alpha \models \beta$ iff $(\alpha \Rightarrow \beta)$ valid

Probability & Statistics Foundation

- **Bayes' rule:** $P(\text{cause}|\text{effect}) \propto P(\text{effect}|\text{cause}) \times P(\text{cause})$
- **Product rule:** $P(a \wedge b) = P(a|b)P(b)$
- **Chain rule:** $P(x_1, \dots, x_n) = \prod P(x_i | x_{i-1}, \dots, x_1)$
- **Law of total probability:** $P(Y) = \sum_z P(Y|z)P(z)$
- **Conditional independence:** $P(X, Y|Z) = P(X|Z)P(Y|Z)$
- **Kolmogorov's axioms:** (1) $0 \leq P(\omega) \leq 1$; (2) $\sum P(\omega) = 1$; (3) $P(a \vee b) = P(a) + P(b) - P(a \wedge b)$
- **Gaussian:** $N(x; \mu, \sigma^2) = (1/\sigma\sqrt{2\pi}) e^{-\frac{1}{2}((x-\mu)/\sigma)^2}$; closed under linear transforms and Bayesian updating
- **Central limit theorem:** sum of many i.i.d. RVs $\approx$ Gaussian
- **Expectation:** $E[X] = \sum x \cdot P(x)$; MEU = maximize expected utility
- **Naive Bayes:** $P(\text{Cause}|e) = \alpha P(\text{Cause}) \prod P(e_i|\text{Cause})$
- **Maximum likelihood:** choose θ maximizing $P(\text{data}|\theta)$
- **Bayesian estimation:** $P(\theta|\text{data}) \propto P(\text{data}|\theta) P(\theta)$
- **Normalization:** $\alpha = 1/P(e)$ ensures posterior sums to 1
- **Inclusion-exclusion:** $P(a \vee b) = P(a) + P(b) - P(a \wedge b)$

People & Dates Master List

Person	Contribution	Year(s)
Aristotle	Syllogisms, categories	384-322 BCE
Thomas Bayes	Bayes' rule	1763 (posth.)
George Boole	Mathematical logic	1847
Gottlob Frege	Quantifiers, FOL	1879
A.A. Markov	Markov chains	1913
Ernst Zermelo	Minimax algorithm	1912
John von Neumann	Game theory, utility	1928-1944
Alan Turing	Computability, Turing test	1936-1950
Claude Shannon	Type A/B chess, n-grams	1950

Person	Contribution	Year(s)
John McCarthy	Lisp, alpha-beta, FOL for AI	1956-1958
Marvin Minsky	Perceptrons, frames	1969-1975
John Alan Robinson	Resolution	1965
Judea Pearl	Bayes nets, heuristics, causal inference	1982-2000
Richard Bellman	Dynamic programming, MDPs	1957
John F. Nash, Jr.	Nash equilibrium	1950
Donald Knuth	Alpha-beta correctness	1975
Lotfi Zadeh	Fuzzy logic	1965
Rudolf Kalman	Kalman filter	1960
Andrew Viterbi	Viterbi algorithm	1967
David Silver	AlphaGo, AlphaZero	2016-2018
Krizhevsky et al.	AlexNet, deep CNN breakthrough	2012
Vaswani et al.	Transformer	2018
Devlin et al.	BERT	2018
Radford et al.	GPT-2	2019

Exam Questions by Type

MCQ

- Q:** Which of the following is NOT one of the four approaches to AI?
 - a) Acting humanly
 - b) Thinking rationally
 - c) Acting efficiently
 - d) Thinking humanly

A: c. **Explanation:** The four are acting humanly, thinking humanly, thinking rationally, acting rationally.
- Q:** A* search is guaranteed to find the optimal solution when:
 - a) The heuristic is consistent
 - b) The heuristic is admissible
 - c) The search space is finite
 - d) Both a and b

A: d. **Explanation:** Consistent implies admissible; with admissible heuristic A* is cost-optimal.
- Q:** Alpha-beta pruning with perfect move ordering reduces the effective branching factor of minimax from:
 - a) b to $\sqrt{b}$
 - b) b to $b/2$
 - c) b^m to $b^{(m/2)}$
 - d) b^m to $\sqrt{b}$

- **A: a. Explanation:** Perfect ordering gives $O(b^{(m/2)}) \rightarrow$ effective branching factor $\approx \sqrt{b}$.
- 4. **Q:** In a CSP, the MRV (Minimum-Remaining-Values) heuristic selects:
 - a) The variable with the most constraints
 - b) The value that rules out fewest choices for neighbors
 - c) The variable with the fewest legal values
 - d) The variable with the highest degree
 - **A: c. Explanation:** MRV = fail-first; choose most constrained variable.
- 5. **Q:** Bayes' rule allows computing $P(\text{cause}|\text{effect})$ from:
 - a) $P(\text{effect}), P(\text{cause}), P(\text{effect}|\text{cause})$
 - b) $P(\text{cause}), P(\text{effect}|\text{cause}), P(\text{effect})$
 - c) $P(\text{cause}|\text{effect}), P(\text{cause}), P(\text{effect})$
 - d) $P(\text{effect}|\text{cause}), P(\text{cause}), P(\text{effect})$
 - **A: d. Explanation:** $P(\text{cause}|\text{effect}) = P(\text{effect}|\text{cause})P(\text{cause})/P(\text{effect})$.
- 6. **Q:** A Bayesian network with all Boolean variables has n nodes, each with at most k parents. How many numbers for the CPTs?
 - a) 2^n
 - b) $n \times 2^k$
 - c) $2^{\{n+k\}}$
 - d) $k \times 2^n$
 - **A: b. Explanation:** Each CPT has $\leq 2^k$ entries; n nodes.
- 7. **Q:** The Viterbi algorithm differs from filtering in that:
 - a) It sums instead of maximizing
 - b) It maximizes instead of summing, and has no normalization constant
 - c) It uses only the sensor model
 - d) It requires a Gaussian distribution
 - **A: b. Explanation:** Viterbi replaces sum with max; no normalization.
- 8. **Q:** In a Bayesian network, the Markov blanket of a node includes:
 - a) Its parents only
 - b) Its children only
 - c) Its parents, children, and children's other parents
 - d) All ancestors
 - **A: c. Explanation:** Markov blanket = parents + children + co-parents.
- 9. **Q:** Which of the following is true about PARTIALLY OBSERVABLE environments for simple reflex agents?
 - a) They work fine with no internal state
 - b) They can get stuck in infinite loops without randomization
 - c) They always find the optimal solution
 - d) They require no percepts
 - **A: b. Explanation:** Simple reflex agents need fully observ-

- able environments; randomization can help escape loops.
10. **Q:** The QUALIFICATION PROBLEM refers to:
 - a) The difficulty of enumerating all preconditions for an action to succeed
 - b) The need to specify time-indexed variables
 - c) The exponential size of truth tables
 - d) The problem of learning from noisy data
 - **A:** a. **Explanation:** Qualification problem: cannot list all conditions (car breakdown, meteorite, etc.).
 11. **Q:** In the Wumpus world, the successor-state axiom for HaveArrow is:
 - a) $\text{HaveArrow}^{t+1} \Leftrightarrow (\text{HaveArrow}^t \vee \text{Shoot}^t)$
 - b) $\text{HaveArrow}^{t+1} \Leftrightarrow (\text{HaveArrow}^t \wedge \neg \text{Shoot}^t)$
 - c) $\text{HaveArrow}^{t+1} \Leftrightarrow (\neg \text{HaveArrow}^t \wedge \text{Shoot}^t)$
 - d) $\text{HaveArrow}^{t+1} \Leftrightarrow (\text{HaveArrow}^t \Rightarrow \neg \text{Shoot}^t)$
 - **A:** b. **Explanation:** Have arrow persists unless Shoot action used.
 12. **Q:** Nash's theorem states that:
 - a) Every game has a dominant strategy equilibrium
 - b) Every game has at least one Nash equilibrium in mixed strategies
 - c) Every zero-sum game has a pure-strategy equilibrium
 - d) Every game has a Pareto optimal outcome
 - **A:** b. **Explanation:** Nash (1950) proved existence of mixed-strategy equilibrium in all finite games.
 13. **Q:** What does the Bellman update operator do?
 - a) Updates policy by evaluating all states
 - b) Updates utilities using max over actions of expected discounted future rewards
 - c) Updates the discount factor
 - d) Updates the transition model
 - **A:** b. **Explanation:** $U(s) \leftarrow \max_a \sum P(s'|s,a)[R(s,a,s') + \gamma U(s')]$.
 14. **Q:** Which of the following is a generative model for NLP?
 - a) Logistic regression
 - b) SVM
 - c) HMM
 - d) Perceptron
 - **A:** c. **Explanation:** HMM models joint $P(W,C)$; logistic regression is discriminative $P(C|W)$.
 15. **Q:** The self-attention mechanism in transformers computes:
 - a) Query, key, value vectors from each input
 - b) Convolution over local windows
 - c) Recurrent hidden states
 - d) Only global average pooling
 - **A:** a. **Explanation:** Self-attention = $q_i = W_q x_i, k_i = W_k$

$$x_i, v_i = W_v x_i; \text{attention scores} = \text{softmax}(q \cdot k / \sqrt{d}).$$

Short Answer

1. **Q:** State Bayes' rule and explain why causal knowledge is more robust than diagnostic knowledge. **Rubric:** $P(b|a) = P(a|b)P(b)/P(a)$. Causal knowledge $P(\text{effect}|\text{cause})$ reflects underlying mechanisms unaffected by changes in priors. Diagnostic knowledge $P(\text{cause}|\text{effect})$ must be re-estimated when priors change (e.g., epidemic raises $P(\text{meningitis})$).
2. **Q:** Define d-separation and explain how to determine if X is conditionally independent of Y given Z . **Rubric:** Consider ancestral subgraph of X, Y, Z ; moralize (add links between unpaired parents sharing child); replace directed with undirected. If Z blocks all paths between X and Y , then $X \perp Y \mid Z$.
3. **Q:** Describe the three inference tasks in temporal models and give the formula for filtering. **Rubric:** Filtering (state estimation): $P(X_t | e_{1:t})$; Prediction: $P(X_{t+k} | e_{1:t})$; Smoothing: $P(X_k | e_{1:t})$ for $k < t$. Filtering: $P(X_{t+1} | e_{1:t+1}) = \alpha P(e_{t+1} | X_{t+1}) \sum P(X_{t+1} | x_t) P(x_t | e_{1:t})$.
4. **Q:** Explain the difference between forward chaining and backward chaining in logic. **Rubric:** Forward chaining: data-driven, starts from known facts, applies rules to derive new facts (breadth-first, complete). Backward chaining: goal-driven, starts from query, works backward to find supporting facts (depth-first, potentially incomplete). Forward good for deriving all consequences; backward good for answering specific queries.
5. **Q:** What is the value alignment problem and why is it difficult? **Rubric:** Ensuring AI system's objectives match human's true preferences. Difficult because: (1) humans may not know their own preferences precisely; (2) preferences are complex and context-dependent; (3) incomplete specification can lead to reward hacking; (4) King Midas problem — getting exactly what we ask for, not what we want.
6. **Q:** Compare and contrast supervised, unsupervised, and reinforcement learning. **Rubric:** Supervised: labeled examples (x, y) , learn mapping. Unsupervised: unlabeled data, find structure (clustering, density estimation). RL: agent interacts with environment, learns from reward signal. Key difference: RL involves sequential decision-making and exploration.
7. **Q:** Explain the four steps of Monte Carlo Tree Search. **Rubric:** (1) Selection: starting at root, choose moves guided by selection policy (e.g., UCT) down to leaf. (2) Expansion: generate new

child of selected node. (3) Simulation: playout from child node using playout policy (not recorded in tree). (4) Back-propagation: update all nodes on path to root with result.

8. **Q:** What is the frame problem and how do successor-state axioms solve it? **Rubric:** Frame problem: need to specify what stays unchanged after an action. Representational: $O(mn)$ frame axioms needed. Successor-state axiom solution: $F^{t+1} \Leftrightarrow \text{ActionCauses}F^t \vee (F^t \wedge \neg \text{ActionCausesNot}F^t)$ — specifies when F changes, everything else persists by default.
9. **Q:** Define the terms “admissible heuristic” and “consistent heuristic” in A^* search. **Rubric:** Admissible: $h(n) \leq h^*(n)$ — never overestimates true cost to goal (optimistic). Consistent: $h(n) \leq c(n,a,n') + h(n')$ — triangle inequality. Every consistent heuristic is admissible but not vice versa. Consistent implies A finds optimal path first time reaching each state.
10. **Q:** What is the difference between minimization and expectation in Viterbi vs. filtering? **Rubric:** Filtering uses sum (marginalization) to compute $P(X_t|e_{1:t})$. Viterbi uses max to find most likely sequence $\text{argmax } P(x_{1:t}|e_{1:t})$. No normalization constant in Viterbi. Both use recursion: filtering sums over previous state; Viterbi maximizes.

Trace / Apply

1. **Input:** Umbrella DBN: $P(R_0)=(0.5,0.5)$; $P(R_t|R_{t-1}): 0.7/0.3$; $P(U_t|R_t): 0.9/0.2$. Observe $U_1=\text{true}$, $U_2=\text{true}$. **Apply filtering** to compute $P(R_2|u_1,u_2)$. **Expected output:** Step 1: $P(R_1) = 0.5 \times \langle 0.7, 0.3 \rangle + 0.5 \times \langle 0.3, 0.7 \rangle = \langle 0.5, 0.5 \rangle$. Step 2: $P(R_1|u_1) = \alpha(0.9, 0.2) \times \langle 0.5, 0.5 \rangle = \alpha(0.45, 0.1) = \langle 0.818, 0.182 \rangle$. Step 3: $P(R_2|u_1) = 0.818 \times \langle 0.7, 0.3 \rangle + 0.182 \times \langle 0.3, 0.7 \rangle = \langle 0.627, 0.373 \rangle$. Step 4: $P(R_2|u_1,u_2) = \alpha(0.9, 0.2) \times \langle 0.627, 0.373 \rangle = \alpha(0.565, 0.075) = \langle 0.883, 0.117 \rangle$.
2. **Input:** Burglary net (Pearl’s example): $P(B)=0.001$, $P(E)=0.002$, $P(A|B,E): 0.95/0.94/0.29/0.001$, $P(J|A): 0.90/0.05$, $P(M|A): 0.70/0.01$. Query $P(B|j,m)$. **Apply variable elimination** to compute $P(B|j,m)$. **Expected output:** $f_4(A)=\langle 0.90, 0.05 \rangle$, $f_5(A)=\langle 0.70, 0.01 \rangle$. Multiply $f_3(A,B,E) \times f_4(A) \times f_5(A)$. Sum out $A \rightarrow f_6(B,E)$. Sum out $E \rightarrow f_7(B)$. Multiply with $f_1(B)=\langle 0.001, 0.999 \rangle$. Normalize $\rightarrow \langle 0.284, 0.716 \rangle$.
3. **Input:** CSP with variables $A\{1,2,3\}$, $B\{1,2,3\}$, $C\{1,2,3\}$, constraints: $A \neq B$, $B \neq C$, $A > C$. **Apply backtracking with MRV + forward checking. Expected output:** Variables have same domain size initially; choose A (arbitrary). Try $A=1 \rightarrow \text{FC: } B \neq 1$,

$C < 1 \rightarrow C = \emptyset \rightarrow$ backtrack. $A = 2 \rightarrow$ FC: $B \neq 2, C < 2 \rightarrow C = \{1\}$. Then B has $\{1, 3\}$, $C = \{1\}$. Assign $B = 1 \rightarrow$ FC: $C \neq 1 \rightarrow C = \emptyset$ (but $C = 1$ and $B \neq C$ fails) $\rightarrow$ backtrack $B = 3 \rightarrow$ success. Solution: $A = 2, B = 3, C = 1$.

4. **Input:** Prisoner's Dilemma payoff matrix: $(C, C) = (-1, -1)$, $(C, D) = (-10, 0)$, $(D, C) = (0, -10)$, $(D, D) = (-5, -5)$. **Find Nash equilibrium.**
Expected output: For each player: if other cooperates, defect gives $0 > -1$; if other defects, defect gives $-5 > -10$. Defect strictly dominates cooperate. Unique Nash: (Defect, Defect) with payoff $(-5, -5)$, even though (Cooperate, Cooperate) = $(-1, -1)$ is Pareto-optimal.
5. **Input:** MDP 4×3 world, $\gamma = 1$, $r = -0.04$ per step. Start $(1, 1)$, goal $(4, 3) = +1$, $(4, 2) = -1$. Transition: 0.8 intended, 0.1 each perpendicular. **Compute one iteration of value iteration for state $(3, 2)$** given initial $U = 0$. **Expected output:** Actions: Up $\rightarrow$ possible s' : $(4, 2)/-1$ (0.8), $(3, 3)/0$ (0.1), $(3, 1)/0$ (0.1) $\rightarrow EU = 0.8(-1) + 0.1(0) + 0.1(0) + (-0.04) = -0.84$. Down $\rightarrow (2, 2)/0$ (0.8), $(3, 3)/0$ (0.1), $(3, 1)/0$ (0.1) $\rightarrow EU = 0 + (-0.04) = -0.04$. Left $\rightarrow (3, 1)/0$ (0.8), $(2, 2)/0$ (0.1), $(4, 2)/-1$ (0.1) $\rightarrow EU = 0.8(0) + 0.1(0) + 0.1(-1) + (-0.04) = -0.14$. Right $\rightarrow (4, 2)/-1$ (0.8), $(3, 3)/0$ (0.1), $(3, 1)/0$ (0.1) $\rightarrow EU = 0.8(-1) + (-0.04) = -0.84$. Max is Down with -0.04 .

Essay / Long-Form

1. **Q:** Compare and contrast HMMs, Kalman filters, and DBNs for temporal probabilistic reasoning. Discuss representation, inference complexity, and when each is appropriate. **Key points:** HMM: single discrete state, $O(S^2)$ per step, atomic state representation; good for speech, bioinformatics. Kalman: continuous vector, linear-Gaussian, $O(n^3)$ per step, Gaussian posterior; good for tracking. DBN: multiple variables, factored representation (linear in n), exact inference exponential in n ; generalizes both. DBN preferred for complex systems with many interacting variables; particle filtering makes DBN inference practical. HMMs suffer state explosion; Kalman fails with nonlinearity.
2. **Q:** Explain the role of conditional independence in scaling probabilistic inference. Discuss with examples from Bayes nets and the naive Bayes model. **Key points:** Full joint distribution $O(2^n)$; conditional independence allows factorization into $O(n2^k)$. Bayes net: each variable independent of non-descendants given parents. Naive Bayes: all effects independent given cause. Wumpus world: frontier variables separate query from irrelevant variables. Without conditional independence, probabilistic AI would be infeasible. The chain rule with Bayes net conditional indepen-

dence: $P(x_1, \dots, x_n) = \prod P(x_i | \text{parents}(X_i))$ instead of $P(x_1, \dots, x_n) = \prod P(x_i | x_1, \dots, x_{i-1})$.

3. **Q:** Discuss the ethics and safety challenges of AI. Address lethal autonomous weapons, bias in ML systems, and the value alignment problem. **Key points:** LAWs: weapons that autonomously select/engage targets; qualitatively different (scalable, no human judgment); debate on banning (UN CCW discussions). ML bias: data reflects societal biases → COMPAS recidivism (different false positive rates across races); fairness criteria conflict (Kleinberg impossibility); need diverse teams, data sheets, subgroup metrics. Value alignment: King Midas problem; reward hacking (gaming reward function); negative side effects; Russell's three principles (maximize revealed preferences, uncertainty about preferences, human behavior as evidence). AI safety: corrigibility, off-switch, cautious behavior under uncertainty.
4. **Q:** Trace the historical development of AI from 1943 to the present, identifying key breakthroughs and periods of progress/retrenchment. **Key points:** 1943 McCulloch-Pitts neurons → 1956 Dartmouth (birth of AI) → Logic Theorist, GPS → Lisp (1958) → microworlds. 1969: Perceptrons (Minsky & Papert) → first AI winter. 1970s: expert systems (DENDRAL, MYCIN, R1). 1980s: second AI winter, back-propagation revival. 1988: Bayes nets (Pearl). 2011+: deep learning revolution (ImageNet 2012, AlphaGo 2016, AlphaZero 2017). Current: transformers, large language models, reinforcement learning breakthroughs. Driving factors: Moore's law (compute), big data, algorithmic advances, GPU/TPU hardware.
5. **Q:** Compare and contrast the complete and approximate inference algorithms for Bayesian networks. When would you use each? **Key points:** Exact: variable elimination (sum out hidden variables from pointwise products of factors), poly tree linear, multiply connected exponential (NP-hard). Approximate: rejection sampling (inefficient with much evidence), likelihood weighting (weights degrade with downstream evidence), Gibbs MCMC (good with downstream evidence, can fail with deterministic CPTs). Exact good for small polytrees; approximate for large networks or when exact infeasible. Variable elimination optimal for polytrees; Gibbs for multiply connected with no deterministic dependencies. Likelihood weighting best when evidence is upstream (near root).